Πανεπιστήμιο Πειραιώς
Τμήμα Ψηφιακών Συστημάτων
Π.Μ.Σ. “Πληροφοριακά Συστήματα και Υπηρεσίες”

ΚΑΤΕΥΘΥΝΣΗ: Μεγάλα Δεδομένα & Αναλυτική

ΜΑΘΗΜΑ: Διαχείριση Δεδομένων για Σχεσιακές και μη Σχεσιακές Βάσεις

ΚΑΘΗΓΗΤΗΣ: Χ. ΔΟΥΛΚΕΡΙΔΗΣ

ΜΕΤΑΠΤΥΧΙΑΚΗ ΑΠΑΛΛΑΚΤΙΚΗ ΕΡΓΑΣΙΑ ΦΕΒ. 2025-26

Σχεδίαση και Υλοποίηση Μη Σχεσιακής ΒΔ στο Σύστημα MongoDB

ΟΜΑΔΑ

Κολοβός Γεώργιος – Τσούτσουρας Θεόδωρος

ΑΜ: ΜΕ25019 - **ΑΜ:** ΜΕ25040

Αθήνα, Δεκέμβρης 2025 – Ιανουάριος 2026

Εισαγωγή

Η ραγδαία αύξηση του όγκου και της πολυπλοκότητας χωροχρονικών δεδομένων που παράγονται από σύγχρονα συστήματα παρακολούθησης, όπως τα συστήματα αυτόματης αναγνώρισης πλοίων (AIS), καθιστά ανεπαρκείς τις παραδοσιακές σχεσιακές βάσεις δεδομένων για αποδοτική αποθήκευση και επεξεργασία. Τα δεδομένα αυτά χαρακτηρίζονται από υψηλή συχνότητα καταγραφής, έντονη χωρική και χρονική διάσταση, καθώς και από ετερογενή και εξελισσόμενη δομή [1], [2]. Στο πλαίσιο αυτό, οι μη σχεσιακές βάσεις δεδομένων προσφέρουν αυξημένη ευελιξία στη μοντελοποίηση, καλύτερη υποστήριξη για χωρικά δεδομένα και δυνατότητες κλιμάκωσης που είναι κρίσιμες για εφαρμογές ανάλυσης μεγάλων χωροχρονικών συνόλων δεδομένων.

Στόχος της παρούσας εργασίας είναι ο σχεδιασμός και η υλοποίηση μιας μη σχεσιακής βάσης δεδομένων στο σύστημα MongoDB για την αποθήκευση και ανάλυση πραγματικών δεδομένων AIS στην ευρύτερη περιοχή του λιμένα Πειραιά. Το σύνολο δεδομένων περιλαμβάνει θέσεις πλοίων, πληροφορία τροχιών, στατικά χαρακτηριστικά πλοίων, καθώς και γεωγραφικά και μετεωρολογικά δεδομένα. Η εργασία επικεντρώνεται στη μοντελοποίηση και τον εμπλουτισμό των τροχιών πλοίων, στη διαδικασία μεταφόρτωσης και ενοποίησης των δεδομένων, καθώς και στην υποστήριξη σύνθετων χωροχρονικών και συναθροιστικών ερωτημάτων.

2. Ανάλυση συνόλου δεδομένων

Η σχεδίαση της βάσης και των διαδικασιών εμπλουτισμού δεν έγινε «στα τυφλά», αλλά καθοδηγήθηκε από μια αρχική επιθεώρηση του συνόλου δεδομένων (EDA) με στόχο: (i) να κατανοηθούν οι δομές/πεδία των αρχείων, (ii) να εντοπιστούν ποιοτικά ζητήματα (ελλείψεις/ασυνέχειες), και (iii) να επιλεγούν με τεκμηριωμένο τρόπο κρίσιμες παράμετροι (thresholds) που επηρεάζουν άμεσα την κατασκευή και τη χρηστικότητα των τροχιών. Το dataset αφορά AIS θέσεις πλοίων στην περιοχή του Σαρωνικού/Πειραιά, με γεωαναφορά σε πολύ μεγάλο όγκο εγγραφών (δεκάδες εκατομμύρια θέσεις).

Για τον λόγο αυτό δημιουργήθηκαν δυο jupyter notebooks (**Inspect_dataset_partA, Inspect_dataset_partB**) για την μελέτη του dataset. Η μεγάλη κλίμακα των AIS points είναι ακριβώς ο λόγος που ζητήθηκε NoSQL προσέγγιση, καθώς επιτρέπει ευέλικτη μοντελοποίηση χωροχρονικών δεδομένων και κλιμάκωση μέσω ευρετηρίων.

2.1 Δομή και χαρακτηριστικά των AIS δεδομένων (δυναμικό σκέλος)

Το αρχείο AIS παρέχει μια ελάχιστη αλλά επαρκή περιγραφή κίνησης ανά χρονική στιγμή, με πεδία όπως t (UNIX timestamp σε milliseconds), vessel_id, lon, lat, heading, course, speed. Η συγκεκριμένη επιλογή πεδίων είναι τυπική για kinematic AIS streams και επιτρέπει να ανακατασκευαστεί η τροχιά ως ακολουθία χωροχρονικών σημείων (space-time points) με πρόσθετα χαρακτηριστικά κίνησης (ταχύτητα/heading) [1], [2]. Σε επίπεδο μετασχηματισμού για MongoDB, τα lon/lat μετατρέπονται σε GeoJSON Point ώστε να υποστηριχθούν χωρικά ερωτήματα με 2dsphere ευρετήριο, ενώ το t μετατρέπεται σε UTC datetime για φυσική υποστήριξη χρονικών φίλτρων και χρονικών παραθύρων.

```
Index(['t', 'vessel_id', 'lon', 'lat', 'heading', 'speed', 'course'], dtype='object')
```

	t	vessel_id	lon	lat	heading	speed	course
0	1548701541000	29b2c240d3ee158281fb4ad556939ee938d08685bd796a...	23.615058	37.944708	NaN	0.0	221.3
1	1548701542000	4370e27814614fc0a8e27e9289a7a9299dc8add830e993...	23.554450	37.875880	141.0	0.3	6.4
2	1548701543000	5de00399a2fdba5f2eb761a622a79a73af9d4058d51e6b...	23.641648	37.944590	NaN	0.0	196.4
3	1548701544000	7f080c68ffb5cbe042d98b2524fc8a56db5bb75315f106...	23.641433	37.947900	179.0	0.0	103.0
4	1548701545000	0fe51bee65c87f2f06f33d5b42fd47dfaf7d99c72336e2...	23.612498	37.897965	175.0	14.5	173.5

```
t min/max: 1548701541000 1548701545000
```

	t	dt
0	1548701541000	2019-01-28 18:52:21+00:00
1	1548701542000	2019-01-28 18:52:22+00:00
2	1548701543000	2019-01-28 18:52:23+00:00
3	1548701544000	2019-01-28 18:52:24+00:00
4	1548701545000	2019-01-28 18:52:25+00:00

Κατά την επιθεώρηση δειγμάτων του dataset διαπιστώθηκε ότι ορισμένα πεδία εμφανίζουν ελλείψεις (π.χ. heading, ενίοτε speed/course), και ότι υπάρχουν εγγραφές που φέρουν ήδη “system annotations” (π.χ. ετικέτες όπως GAP_END). Το εύρημα αυτό επηρέασε ώστε να εισάγουμε δικές μας ετικέτες (annotations) σε επίπεδο σημείου, οι οποίες είναι σταθερά ορισμένες και προκύπτουν από κανόνες πάνω στα πεδία κίνησης.

2.2 Ποιοτική εικόνα και γιατί χρειάζονται thresholds

Πριν την κατασκευή των τροχιών, απαιτείται να οριστούν βασικές έννοιες όπως τι θεωρείται στάση, τροχια ή αλλαγή πορείας ενός πλοίου. Η ανάγκη αυτή προκύπτει από τα ίδια τα δεδομένα. Κατά την αρχική επιθεώρηση του συνόλου AIS, παρατηρείται ότι η ταχύτητα των πλοίων παρουσιάζει μεγάλη διακύμανση, ενώ ένα σημαντικό ποσοστό εγγραφών έχει μηδενική ή πολύ χαμηλή τιμή ταχύτητας. Το φαινόμενο αυτό είναι αναμενόμενο σε λιμενική περιοχή, όπου τα πλοία συχνά παραμένουν ακίνητα ή κινούνται πολύ αργά κατά τη διάρκεια αναμονών και ελιγμών.

	t	vessel_id	lon	lat	heading	speed	course
count	5.000000e+03	5000	5000.000000	5000.000000	2973.000000	4992.000000	4558.000000
unique	NaN	115	NaN	NaN	NaN	NaN	NaN
top	NaN	ae75f098caecc8c85bff774f4278e2798cf984876d656e...	NaN	NaN	NaN	NaN	NaN
freq	NaN	395	NaN	NaN	NaN	NaN	NaN
mean	1.548703e+12	NaN	23.612844	37.928985	173.617894	3.126783	169.497762
std	8.718335e+05	NaN	0.042237	0.033677	88.927648	5.913058	109.818432
min	1.548702e+12	NaN	23.391842	37.777733	0.000000	0.000000	0.000000
25%	1.548702e+12	NaN	23.588735	37.930017	153.000000	0.000000	62.925000
50%	1.548703e+12	NaN	23.615018	37.940158	178.000000	0.100000	181.000000
75%	1.548704e+12	NaN	23.641628	37.948137	214.000000	2.200000	263.300000
max	1.548705e+12	NaN	23.683927	38.029410	359.000000	20.700000	359.500000

Επιπλέον, διαπιστώνεται ότι πεδία όπως το course και το heading δεν είναι πάντα διαθέσιμα, γεγονός που καθιστά απαραίτητο οι κανόνες ανάλυσης να είναι ανθεκτικοί σε ελλιπή δεδομένα. Για τους παραπάνω λόγους, επιλέχθηκε η χρήση thresholds που μετατρέπουν συνεχείς τιμές (όπως ταχύτητα και γωνιακή μεταβολή πορείας) σε διακριτές

ετικέτες κίνησης. Η επιλογή αυτή απλοποιεί σημαντικά τη διατύπωση των ερωτημάτων και επιτρέπει την αποδοτική ευρετηρίαση πάνω σε annotations, ιδιαίτερα όταν τα φίλτρα συνδυάζουν χωρικά, χρονικά και συμπεριφορικά κριτήρια [1], [2].

```
#  metraw poso syxna leipei kathe pedio (NaN rate)
missing_rate = dyn.isna().mean().sort_values(ascending=False)
missing_rate.head(15)
```

```
heading      0.4054
course       0.0884
speed        0.0016
t            0.0000
lat          0.0000
lon          0.0000
vessel_id    0.0000
dt_utc       0.0000
dtype: float64
```

2.3 Κατάτμηση σε τροχιές: αιτιολόγηση του χρονικού gap (GAP_HOURS)

Η κατασκευή τροχιών από μεμονωμένα AIS points απαιτεί την ύπαρξη κανόνα κατάτμησης, καθώς το AIS data δεν είναι συνεχές. Στην πράξη, εμφανίζονται χρονικά κενά πιθανών λόγω απώλειας σήματος, απενεργοποίησης συσκευών ή απομάκρυνσης των πλοίων από την περιοχή κάλυψης. Για τον λόγο αυτό, στην υλοποίηση **μια νέα τροχιά ξεκινά είτε όταν αλλάζει το vessel_id, είτε όταν η χρονική απόσταση από το προηγούμενο σημείο του ίδιου πλοίου υπερβαίνει ένα προκαθορισμένο χρονικό όριο (GAP_HOURS).**

```
# kanw sort ana ploio + xrono kai vlepo ta time gaps (gia na apofasisw threshold)
tmp = dyn.dropna(subset=["dt_utc"]).sort_values([vessel_col, "dt_utc"]).copy()

v = tmp[vessel_col].value_counts().index[0]  # top vessel sto sample
one = tmp[tmp[vessel_col] == v].copy()
one["gap_min"] = one["dt_utc"].diff().dt.total_seconds() / 60

one[["dt_utc", "gap_min"]].head(25), one["gap_min"].describe()
```

```
(                      dt_utc   gap_min
 11  2019-01-28 18:52:29+00:00       NaN
 25  2019-01-28 18:52:38+00:00  0.150000
 45  2019-01-28 18:52:49+00:00  0.183333
 63  2019-01-28 18:52:59+00:00  0.166667
 79  2019-01-28 18:53:09+00:00  0.166667
 91  2019-01-28 18:53:17+00:00  0.133333
 140 2019-01-28 18:53:47+00:00  0.500000
 157 2019-01-28 18:53:59+00:00  0.200000
 177 2019-01-28 18:54:08+00:00  0.150000
 190 2019-01-28 18:54:17+00:00  0.150000
 216 2019-01-28 18:54:29+00:00  0.200000
 232 2019-01-28 18:54:38+00:00  0.150000
 245 2019-01-28 18:54:47+00:00  0.150000
 265 2019-01-28 18:54:59+00:00  0.200000
 290 2019-01-28 18:55:17+00:00  0.300000
 312 2019-01-28 18:55:29+00:00  0.200000
 327 2019-01-28 18:55:38+00:00  0.150000
 338 2019-01-28 18:55:47+00:00  0.150000
 350 2019-01-28 18:55:59+00:00  0.200000
 365 2019-01-28 18:56:08+00:00  0.150000
 374 2019-01-28 18:56:17+00:00  0.150000
 393 2019-01-28 18:56:29+00:00  0.200000
 420 2019-01-28 18:56:47+00:00  0.300000
 459 2019-01-28 18:57:08+00:00  0.350000
 470 2019-01-28 18:57:17+00:00  0.150000,
```

Η χρήση χρονικού gap αποτελεί καθιερωμένη πρακτική στην ανάλυση τροχιών, καθώς μεγάλα χρονικά κενά δεν πρέπει να «γεφυρώνονται». Η γεφύρωση τέτοιων κενών θα οδηγούσε σε ψευδείς τροχιές, όπου θα υπονοείται ομαλή κίνηση του πλοίου σε χρονικά διαστήματα για τα οποία δεν υπάρχει καμία πληροφορία. Το επιλεγμένο όριο επιδιώκει να

ισορροπήσει μεταξύ της υπερβολικής κατάτμησης σε πολύ μικρές τροχιές και της ένωσης τμημάτων που πιθανόν αντιστοιχούν σε διαφορετικές επιχειρησιακές φάσεις. Επιπλέον, η ύπαρξη ενδείξεων χρονικών κενών στο αρχικό σύνολο δεδομένων ενισχύει τη σχεδιαστική επιλογή να θεωρείται το μεγάλο χρονικό κενό ως φυσικό όριο τροχιάς.

2.4 Avnotations κίνησης: αιτιολόγηση thresholds ταχύτητας και στροφής

Για να υποστηριχθούν αποδοτικά ερωτήματα «με βάση annotations» (όπως ζητά η απαίτηση A2), σε κάθε σημείο παράγεται annotation. Η παραγωγής τους ακολουθεί σαφείς κανόνες: (i) STOP/SLOW/NORMAL/FAST με βάση την ταχύτητα, (ii) MISSING_HEADING όταν λείπει heading, (iii) MISSING_COURSE όταν λείπει course, (iv) TURN όταν η γωνιακή απόκλιση από το προηγούμενο course υπερβαίνει συγκεκριμένο όριο (TURN_THRESHOLD_DEG).

	t	vessel_id	lon	lat	heading	speed	course
count	5.000000e+03	5000	5000.000000	5000.000000	2973.000000	4992.000000	4558.000000
unique	NaN	115	NaN	NaN	NaN	NaN	NaN
top	NaN	ae75f098caecc8c85bff774f4278e2798cf984876d656e...	NaN	NaN	NaN	NaN	NaN
freq	NaN	395	NaN	NaN	NaN	NaN	NaN
mean	1.548703e+12	NaN	23.612844	37.928985	173.617894	3.126783	169.497762
std	8.718335e+05	NaN	0.042237	0.033677	88.927648	5.913058	109.818432
min	1.548702e+12	NaN	23.391842	37.777733	0.000000	0.000000	0.000000
25%	1.548702e+12	NaN	23.588735	37.930017	153.000000	0.000000	62.925000
50%	1.548703e+12	NaN	23.615018	37.940158	178.000000	0.100000	181.000000
75%	1.548704e+12	NaN	23.641628	37.948137	214.000000	2.200000	263.300000
max	1.548705e+12	NaN	23.683927	38.029410	359.000000	20.700000	359.500000

Ταχύτητα (STOP/SLOW/NORMAL/FAST). Η παρουσία πολλών χαμηλών/μηδενικών τιμών ταχύτητας στο dataset καθιστά πρακτικό να οριστεί ένα μικρό όριο για STOP (π.χ. ≤ 0.5 ή ≤ 1 κόμβο ανάλογα με την επιλογή), ώστε να διαχωριστούν οι πραγματικές στάσεις/σχεδόν στάσεις από τις αργές κινήσεις ελιγμών. Το επόμενο εύρος (SLOW) αποτυπώνει τυπική «λιμενική» κίνηση όπου το σκάφος κινείται αλλά δεν ταξιδεύει με ταχύτητα πλεύσης, ενώ το FAST χρησιμοποιείται για να εντοπιστούν σπάνια αλλά ενδιαφέροντα γεγονότα (π.χ. υψηλή ταχύτητα κοντά σε λιμάνι ή περιοχές περιορισμών).

Στροφές (TURN). Για τον εντοπισμό αλλαγών πορείας, η υλοποίηση υπολογίζει τη μικρότερη γωνιακή διαφορά μεταξύ διαδοχικών course τιμών, λαμβάνοντας υπόψη το κυκλικό πεδίο 0–360° (π.χ. 350°→10° είναι 20° και όχι 340°).

Αυτή η στρατηγική “annotation-first” είναι ιδιαίτερα χρήσιμη για τη MongoDB, επειδή επιτρέπει να γράψουμε queries τύπου «βρες τροχιές που περνούν από περιοχή X, σε χρόνο T, με τουλάχιστον N σημεία STOP ή TURN, και με συγκεκριμένες μετεωρολογικές συνθήκες», χωρίς να επαναλαμβάνουμε βαριές αριθμητικές συνθήκες κάθε φορά. Με άλλα λόγια, μέρος της υπολογιστικής προσπάθειας μεταφέρεται στο ingestion/build time (offline), ώστε τα online queries να είναι γρηγορότερα

2.5 Εμπλουτισμός με μετεωρολογικά δεδομένα: χρονικό/χωρικό παράθυρο και caching

Η εργασία απαιτεί «εμπλουτισμένες τροχιές» με μετεωρολογική πληροφορία ανά σημείο. Η αντιστοίχιση AIS σημείου με weather παρατήρηση δεν είναι 1-προς-1: τα weather δεδομένα έχουν διαφορετική χρονική ανάλυση, διαφορετική χωρική θέση, και δεν υπάρχει

κοινό κλειδί. Για αυτό χρησιμοποιείται αντιστοίχιση μέσω χωροχρονικού παραθύρου: για κάθε AIS point γίνεται αναζήτηση του κοντινότερου (geospatial $near) weather record εντός ακτίνας WEATHER_MAX_DISTANCE_METERS και εντός χρονικού παραθύρου ± WEATHER_WINDOW_HOURS.

Ωστόσο, το να εκτελείται ένα τέτοιο geospatial+time query για κάθε AIS σημείο είναι ακριβό, ειδικά σε εκατομμύρια σημεία. Για το λόγο αυτό εφαρμόζεται caching με bucketing σε χώρο και χρόνο: το κλειδί cache κατασκευάζεται από (α) χρονικό bucket π.χ. ανά 3 ώρες και (β) χωρικό bucket π.χ. ανά ~1km (μέσω lon/lat). Έτσι, κοντινά σημεία μέσα στο ίδιο χρονικό/χωρικό «κελί» μοιράζονται το ίδιο αποτέλεσμα weather enrichment, άρα μειώνεται δραστικά ο αριθμός ερωτημάτων προς τη weather συλλογή. Η υλοποίηση αποθηκεύει στο cache ακόμη και το "None" όταν δεν βρίσκεται αποτέλεσμα, ώστε να αποφευχθεί επανάληψη αποτυχημένων queries. Η συγκεκριμένη απόφαση είναι κεντρική για την κλιμακωσιμότητα του build step και μπορεί να παρουσιαστεί ως βελτιστοποίηση τύπου "memoization" πάνω σε χωροχρονικά κελιά [5].

Επιπλέον, κατά το weather lookup εφαρμόζεται projection ώστε να επιστρέφονται μόνο τα πεδία που χρειάζονται για τα ζητούμενα ερωτήματα (π.χ. θερμοκρασία, άνεμος κ.λπ.), μειώνοντας το payload και το κόστος μεταφοράς/αποθήκευσης. Αυτό είναι επίσης σημαντικό για MongoDB λόγω ορίου 16MB ανά έγγραφο: όσο μικρότερο το per-point embedded payload, τόσο περισσότερα σημεία μπορούμε να κρατήσουμε ανά trajectory document πριν φτάσουμε το όριο.

2.6 Στατικό σκέλος πλοίων και «enrichment μία φορά ανά τροχιά»

Το dataset περιλαμβάνει και στατική πληροφορία (π.χ. χώρα, τύπος πλοίου). Η επιθεώρηση των static αρχείων δείχνει ότι είναι μικρής κλίμακας (χιλιάδες εγγραφές) σε σχέση με τα δυναμικά σημεία. Αυτό επιτρέπει να κρατάμε τη static πληροφορία σε ξεχωριστή συλλογή και να την «ενώνουμε» με τις τροχιές. Στην υλοποίηση, το enrichment (country/shiptype) γίνεται μία φορά κατά την έναρξη κάθε νέας τροχιάς και όχι ανά AIS σημείο, ώστε να αποφευχθούν εκατομμύρια επαναλαμβανόμενα lookups. Η επιλογή αυτή τεκμηριώνει καθαρά query-driven σκέψη: η τροχιά είναι το βασικό αντικείμενο ανάλυσης, άρα τα vessel χαρακτηριστικά πρέπει να είναι άμεσα διαθέσιμα στο trajectory document χωρίς να επιβαρύνουν το build/queries.

2.7 Συνοπτικά χωρικά χαρακτηριστικά τροχιάς: bbox/centroid και σύνδεση με ερωτήματα

Για κάθε τροχιά υπολογίζονται συνοπτικά χωρικά χαρακτηριστικά (bounding box και centroid) από τις συντεταγμένες των σημείων. Στη MongoDB το centroid αποθηκεύεται ως GeoJSON Point και ευρετηριάζεται ως 2dsphere, ενώ το bbox μπορεί να χρησιμοποιηθεί για γρήγορο προ-φιλτράρισμα σε εύρη lon/lat. Αυτή η επιλογή σχετίζεται άμεσα με την A2: για χωρικά φίλτρα πάνω σε τροχιές, ένα πρώτο στάδιο μπορεί να ανακτήσει υποψήφιες τροχιές γρήγορα (μέσω centroid/bbox).

2.8 Περιορισμοί και επιπτώσεις σχεδίασης: 16MB έγγραφο και μεγάλα trajectories

Ένας πρακτικός περιορισμός της MongoDB είναι το μέγιστο μέγεθος εγγράφου (16MB). Κατά την κατασκευή τροχιών, μεγάλα trajectories μπορεί να υπερβούν το όριο, ειδικά αν περιέχουν χιλιάδες σημεία και εμπλουτισμό weather ανά σημείο. Για αυτό υπάρχει ρητός χειρισμός αποτυχίας εισαγωγής: αν ένα trajectory document είναι "DocumentTooLarge", καταγράφεται σε ξεχωριστή συλλογή αποτυχιών μαζί με

μεταδεδομένα (vessel_id, χρονικά όρια, πλήθος σημείων). Παρότι αυτός ο μηχανισμός είναι κυρίως αμυντικός, η ύπαρξή του τεκμηριώνει ότι ο σχεδιασμός έλαβε υπόψη περιορισμούς της πλατφόρμας και ότι υπάρχει σαφής στρατηγική καταγραφής/αντιμετώπισης ακραίων περιπτώσεων. Επιπλέον, μπορεί να αναφερθεί ως μελλοντική επέκταση (π.χ. διάσπαση μεγάλων trajectories σε segments ή αποθήκευση points σε ξεχωριστή συλλογή), χωρίς να απαιτείται να υλοποιηθεί πλήρως στο πλαίσιο της εργασίας.

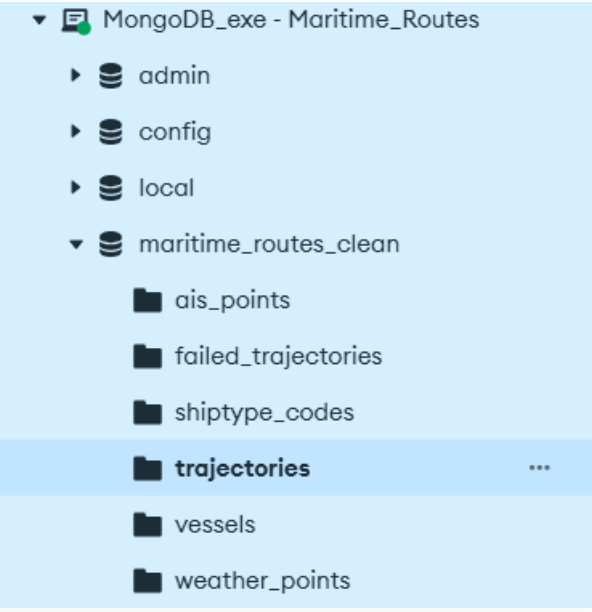
3. Περιβάλλον υλοποίησης και σύνολο δεδομένων που χρησιμοποιήθηκε

Για την υλοποίηση και αξιολόγηση της εργασίας χρησιμοποιήθηκε περιβάλλον βασισμένο σε Docker, ώστε η εγκατάσταση και η εκτέλεση της MongoDB να είναι πλήρως αναπαραγώγιμες και ανεξάρτητες από το λειτουργικό σύστημα. Συγκεκριμένα, η MongoDB εκτελείται σε Docker container με εικόνα **mongo:7**. Στο σύστημά μας χρησιμοποιήθηκε το container με όνομα **mongo-maritime-clean**, το οποίο εκκινείται με την εντολή **docker start mongo-maritime-clean** και είναι διαθέσιμο στο host μέσω port mapping **29018 → 27017** (δηλ. 0.0.0.0:29018 -> 27017). Με τον τρόπο αυτό η βάση δεδομένων είναι προσβάσιμη τοπικά μέσω της διεύθυνσης localhost:29018, χωρίς να απαιτείται εγκατάσταση της MongoDB απευθείας στο λειτουργικό σύστημα.

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
●	mongo-maritime-clean	8e905a4b734d	mongo:7	29018:27017	1.31%	4 minutes ago	
○	mongo-accidents	c82d62fd38d7	mongo:7	29017:27017	0%	5 days ago	

```
PS C:\Users\georg\Desktop\MongoDB_exe - MaritimeRoutes - Final> docker start mongo-maritime-clean
mongo-maritime-clean
PS C:\Users\georg\Desktop\MongoDB_exe - MaritimeRoutes - Final> docker ps -a
CONTAINER ID   IMAGE                       COMMAND                  CREATED       STATUS                  PO
RTS                                        NAMES
8e905a4b734d   mongo:7                     "docker-entrypoint.s…"   11 days ago   Up 6 seconds            0.
0.0.0:29018->27017/tcp, [::]:29018->27017/tcp   mongo-maritime-clean
```

Για τη διαχείριση και την οπτική επιθεώρηση των δεδομένων χρησιμοποιήθηκε το εργαλείο **MongoDB Compass**, το οποίο επέτρεψε τον έλεγχο της δομής των εγγράφων, τη διαχείριση των συλλογών και την επαλήθευση των ευρετηρίων και ερωτημάτων. Η βάση δεδομένων της εργασίας ονομάζεται **maritime_routes_clean** και περιλαμβάνει συλλογές που αντιστοιχούν στα διαφορετικά είδη δεδομένων και στα αποτελέσματα της επεξεργασίας. Συγκεκριμένα, χρησιμοποιούνται οι συλλογές **ais_points** (δυναμικές θέσεις πλοίων), **vessels** και **shiptype_codes** (στατικά χαρακτηριστικά και τύποι πλοίων), **weather_points** (μετεωρολογικές παρατηρήσεις), καθώς και η συλλογή **trajectories**, η οποία περιέχει τις τελικές εμπλουτισμένες τροχιές πλοίων.



Επιπλέον, δημιουργήθηκε η συλλογή **failed_trajectories**, στην οποία καταγράφονται περιπτώσεις όπου η εισαγωγή μιας τροχιάς δεν ήταν δυνατή (π.χ. λόγω υπέρβασης του μέγιστου επιτρεπτού μεγέθους εγγράφου στη MongoDB). Η συλλογή αυτή χρησιμοποιείται για λόγους ιχνηλασιμότητας και τεκμηρίωσης ακραίων περιπτώσεων, χωρίς απώλεια πληροφορίας από τη διαδικασία κατασκευής των τροχιών.

Η φόρτωση και επεξεργασία των δεδομένων πραγματοποιήθηκε μέσω των scripts εισαγωγής και κατασκευής, ενώ για λόγους πρακτικότητας το σύνολο δεδομένων περιορίστηκε χρονικά σε επιλεγμένο υποσύνολο (π.χ. Ιανουάριο 2019), διατηρώντας ωστόσο ρεαλιστικό όγκο και πολυπλοκότητα για την αξιολόγηση της αποδοτικότητας των ερωτημάτων και των σχεδιαστικών επιλογών.

4. Μοντελοποίηση και Σχήμα Βάσης Δεδομένων

Η μοντελοποίηση της βάσης δεδομένων πραγματοποιήθηκε με γνώμονα την αποδοτική υποστήριξη χωροχρονικών ερωτημάτων πάνω σε τροχιές πλοίων, καθώς και τον εμπλουτισμό τους με ετερογενή πληροφορία (στατικά χαρακτηριστικά και μετεωρολογικά δεδομένα). Αντί της άμεσης αποθήκευσης μεμονωμένων AIS σημείων ως βασική μονάδα ανάλυσης, επιλέχθηκε μια **trajectory-centric προσέγγιση**, όπου η τροχιά αποτελεί το κύριο αντικείμενο της βάσης. Η επιλογή αυτή ευθυγραμμίζεται με τα ζητούμενα ερωτήματα της εργασίας και μειώνει το κόστος επαναλαμβανόμενων συνενώσεων (joins) κατά την εκτέλεση σύνθετων αναζητήσεων. Η βάση δεδομένων **maritime_routes_clean** οργανώνεται σε επιμέρους συλλογές, καθεμία από τις οποίες εξυπηρετεί συγκεκριμένο ρόλο στη διαδικασία εισαγωγής, εμπλουτισμού και ανάλυσης των δεδομένων.

4.1 Συλλογή ais_points (δυναμικά δεδομένα θέσης)

Η συλλογή **ais_points** περιέχει τις αρχικές δυναμικές καταγραφές θέσης πλοίων, όπως προκύπτουν από το AIS σύστημα. Κάθε έγγραφο αντιστοιχεί σε μία χρονική στιγμή και περιλαμβάνει το αναγνωριστικό του πλοίου (vessel_id), χρονική πληροφορία (timestamp) και

χωρική θέση αποθηκευμένη σε μορφή GeoJSON Point. Η συλλογή αυτή χρησιμοποιείται κυρίως ως ενδιάμεσο στάδιο για την κατασκευή των τροχιών και όχι για άμεση εκτέλεση ερωτημάτων από τον τελικό χρήστη.

Η επιλογή διατήρησης των raw AIS σημείων σε ξεχωριστή συλλογή επιτρέπει:

- την επαναληψιμότητα της διαδικασίας κατασκευής τροχιών,
- την ευελιξία στην αλλαγή παραμέτρων (π.χ. thresholds),
- και την αποφυγή απώλειας πρωτογενούς πληροφορίας.

4.1.1 Μεταφόρτωση και προεπεξεργασία δυναμικών AIS σημείων (ais_points)

Η συλλογή **ais_points** περιέχει τις πρωτογενείς δυναμικές καταγραφές AIS και λειτουργεί ως ενδιάμεσο στάδιο για την κατασκευή των τελικών τροχιών. Η μεταφόρτωση υλοποιήθηκε με Python (pandas + PyMongo) με στόχο (i) την αποδοτική εισαγωγή μεγάλου CSV, (ii) την κανονικοποίηση των χρονικών και χωρικών πεδίων και (iii) τη δημιουργία κατάλληλων ευρετηρίων για τις επόμενες φάσεις επεξεργασίας [3].

Ανάγνωση σε τμήματα (chunked loading).

Το αρχείο **unipi_ais_dynamic_jan2019.csv** διαβάζεται τμηματικά (chunks) μεγέθους 200.000 γραμμών, ώστε να αποφεύγεται η φόρτωση ολόκληρου του αρχείου στη μνήμη. Για κάθε chunk κατασκευάζεται μία λίστα εγγράφων, η οποία εισάγεται με insert_many σε unordered mode (ordered=False) για καλύτερη απόδοση κατά τη μαζική εισαγωγή.

```
chunk_size = 200_000
for chunk in pd.read_csv(csv_path, chunksize=chunk_size):
    docs = []
    for row in chunk.itertuples(index=False):
        docs.append({...})
    coll.insert_many(docs, ordered=False)
```

Κανονικοποίηση χρόνου (t → timestamp).

Στο αρχικό dataset, ο χρόνος αποθηκεύεται ως epoch timestamp σε milliseconds (t). Για να υποστηριχθούν απευθείας χρονικά φίλτρα και ταξινομήσεις στη MongoDB, το πεδίο μετατρέπεται σε datetime UTC και αποθηκεύεται ως timestamp.

```
def ms_to_dt_utc(ms: int) -> datetime:
    return datetime.fromtimestamp(ms / 1000.0, tz=timezone.utc)
```

```
"timestamp": ms_to_dt_utc(int(row.t))
```

Κανονικοποίηση χώρου (lon/lat → GeoJSON Point).

Οι γεωγραφικές συντεταγμένες (lon, lat) μετατρέπονται σε GeoJSON Point στο πεδίο loc. Αυτό είναι απαραίτητο ώστε να μπορεί να χρησιμοποιηθεί 2dsphere ευρετήριο και να εκτελούνται αποδοτικά χωρικά ερωτήματα (π.χ. $geoWithin, $near) στα επόμενα στάδια[3].

```
"loc": {"type": "Point", "coordinates": [float(row.lon),
float(row.lat)]}
```

Χειρισμός ελλιπών τιμών (NaN → null).

Τα πεδία κίνησης (speed, heading, course) στο dataset μπορεί να περιέχουν ελλείψεις. Για λόγους συνέπειας και για να είναι τα δεδομένα συμβατά με queries/aggregations, οι τιμές NaN μετατρέπονται σε null (δηλ. None στην Python). Έτσι, οι περιπτώσεις έλλειψης πληροφορίας διατηρούνται ρητά και μπορούν να αξιοποιηθούν και ως επισημάνσεις (π.χ. MISSING_COURSE) σε επόμενο στάδιο.

```
"speed": None if pd.isna(row.speed) else float(row.speed),
"heading": None if pd.isna(row.heading) else float(row.heading),
"course": None if pd.isna(row.course) else float(row.course),
```

Δημιουργία ευρετηρίων.

Μετά την εισαγωγή, δημιουργούνται τρία βασικά ευρετήρια, τα οποία υποστηρίζουν τόσο χωρικά όσο και χρονικά workloads:

1. **2dsphere στο loc**, για χωρικά φίλτρα/αναζητήσεις:

```
coll.create_index([("loc", "2dsphere")])
```

2. **Απλό index στο timestamp**, για χρονικά range queries:

```
coll.create_index([("timestamp", ASCENDING)])
```

3. **Σύνθετο index (vessel_id, timestamp)**, για γρήγορη ανάκτηση των σημείων ενός πλοίου σε χρονική σειρά (βασικό για το build των τροχιών):

```
coll.create_index([("vessel_id", ASCENDING), ("timestamp",
ASCENDING)])
```

Με αυτόν τον τρόπο, η συλλογή ais_points δεν αποθηκεύει απλώς τα αρχικά δεδομένα, αλλά παρέχει ήδη την απαιτούμενη μορφή (UTC χρόνο + GeoJSON χώρο) και τα κατάλληλα indexes ώστε η επόμενη φάση (κατασκευή τροχιών και εμπλουτισμός) να μπορεί να εκτελεστεί αποδοτικά και κλιμακώσιμα.

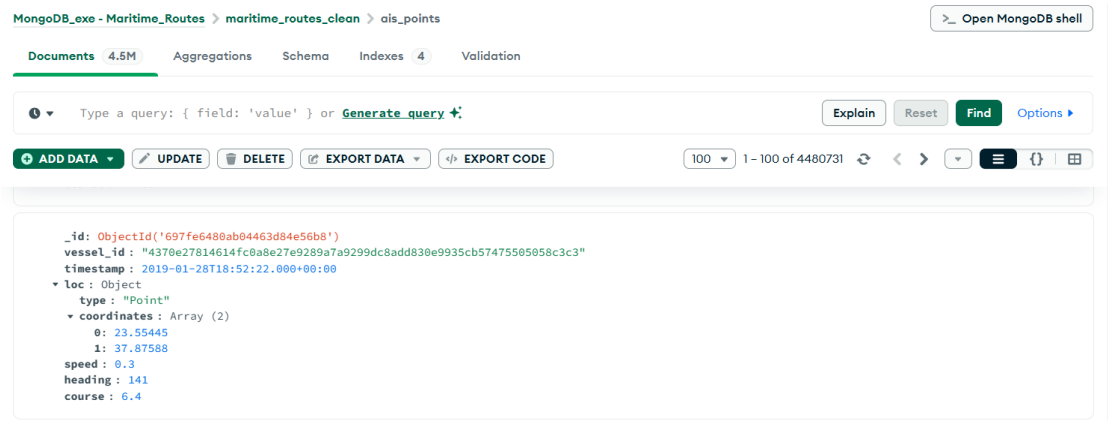
4.1.2 Παράδειγμα εγγράφου ais_points και επιλογές αναπαράστασης

Κάθε έγγραφο στη συλλογή **ais_points** αντιστοιχεί σε μία μεμονωμένη καταγραφή AIS για ένα συγκεκριμένο πλοίο και χρονική στιγμή. Το έγγραφο περιλαμβάνει (i) το αναγνωριστικό πλοίου (vessel_id), (ii) τον χρόνο σε μορφή UTC datetime (timestamp), (iii) τη χωρική θέση σε GeoJSON Point (loc), και (iv) βασικά κινηματικά χαρακτηριστικά (speed, heading, course). Η επιλογή της GeoJSON αναπαράστασης επιτρέπει την απευθείας χρήση 2dsphere ευρετηρίου και χωρικών τελεστών της MongoDB, ενώ η χρήση datetime διευκολύνει χρονικά φίλτρα και ταξινομήσεις.

Παρακάτω φαίνεται ένα ενδεικτικό έγγραφο AIS όπως αποθηκεύεται στη MongoDB:

{
 "_id": "697fe6480ab04463d84e56b8",
 "vessel_id":
"4370e27814614fc0a8e27e9289a7a9299dc8add830e9935cb57475505058c3c3",
 "timestamp": "2019-01-28T18:52:22.000Z",
 "loc": {
  "type": "Point",
  "coordinates": [23.55445, 37.87588]
 },
 "speed": 0.3,
 "heading": 141,
 "course": 6.4
}

Το συγκεκριμένο σχήμα (ένα έγγραφο ανά AIS σημείο) είναι κατάλληλο για αποθήκευση των πρωτογενών δεδομένων και για αποδοτική ανάκτηση σημείων ανά πλοίο και χρονικό διάστημα, ενώ παράλληλα επιτρέπει geospatial ερωτήματα πάνω στα σημεία. Στη συνέχεια, τα AIS points χρησιμοποιούνται ως είσοδος για την κατασκευή των τελικών εμπλουτισμένων τροχιών στη συλλογή trajectories, όπου τα σημεία ομαδοποιούνται και εμπλουτίζονται με annotations και μετεωρολογικά δεδομένα.



4.2 Συλλογές στατικών δεδομένων (vessels, shiptype_codes)

Τα στατικά χαρακτηριστικά των πλοίων αποθηκεύονται στη συλλογή **vessels**, όπου κάθε έγγραφο αντιστοιχεί σε ένα πλοίο και περιλαμβάνει πληροφορίες όπως χώρα και τύπο πλοίου. Η συλλογή **shiptype_codes** λειτουργεί ως βοηθητική δομή αντιστοίχισης κωδικών τύπων πλοίων σε περιγραφικές κατηγορίες.

Η στατική πληροφορία διατηρείται σε ξεχωριστές συλλογές, καθώς:

- είναι μικρού όγκου σε σχέση με τα δυναμικά δεδομένα,
- δεν μεταβάλλεται συχνά,
- και μπορεί να εμπλουτίσει πολλαπλές τροχιές του ίδιου πλοίου.

Κατά την κατασκευή των τροχιών, τα βασικά στατικά χαρακτηριστικά ενσωματώνονται μία φορά σε επίπεδο τροχιάς, ώστε να αποφεύγονται επαναλαμβανόμενα lookups κατά την εκτέλεση ερωτημάτων.

4.2.1 Μεταφόρτωση στατικών δεδομένων πλοίων και τύπων πλοίων (vessels, shiptype_codes)

Τα στατικά χαρακτηριστικά των πλοίων και οι κωδικοί τύπων πλοίων εισάγονται στη βάση δεδομένων μέσω ξεχωριστών συλλογών, ώστε να διαχωρίζονται καθαρά από τα δυναμικά δεδομένα κίνησης. Η διαδικασία υλοποιείται με χρήση pandas και PyMongo και περιλαμβάνει τόσο τον καθαρισμό των δεδομένων όσο και τη δημιουργία ευρετηρίων για αποδοτικό lookup.

Συλλογή shiptype_codes.

Το αρχείο **ais_codes_descriptions.csv** περιέχει αντιστοιχίσεις κωδικών τύπων πλοίων σε περιγραφικές κατηγορίες. Τα δεδομένα εισάγονται απευθείας ως έγγραφα στη συλλογή shiptype_codes, αφού προηγηθεί ένας ελαφρύς καθαρισμός των ονομάτων στηλών. Εφόσον υπάρχει το πεδίο code, δημιουργείται απλό ευρετήριο για γρήγορη αντιστοίχιση τύπων πλοίων.

```
codes_df = pd.read_csv(AIS_CODES)
codes_df.columns = [c.strip() for c in codes_df.columns]
codes_docs = codes_df.to_dict("records")
codes.insert_many(codes_docs, ordered=False)
codes.create_index([("code", ASCENDING)])
```

Η συλλογή αυτή λειτουργεί ως βοηθητικός πίνακας αναφοράς (lookup table) και δεν μεταβάλλεται συχνά, γεγονός που δικαιολογεί τη διατήρησή της σε ξεχωριστή συλλογή.

Συλλογή vessels.

Το αρχείο unipi_ais_static.csv περιέχει στατική πληροφορία για τα πλοία (π.χ. χώρα, τύπος πλοίου). Πριν την εισαγωγή, τα δεδομένα καθαρίζονται και αποδιπλασιάζονται με βάση το vessel_id, ώστε να αποφευχθούν συγκρούσεις μοναδικότητας κατά την εισαγωγή στη MongoDB.

```
static_df = pd.read_csv(AIS_STATIC)
static_df["vessel_id"] = static_df["vessel_id"].astype(str)
static_df = static_df.drop_duplicates(subset=["vessel_id"],
keep="first")
static_df = static_df.where(pd.notna(static_df), None)
```

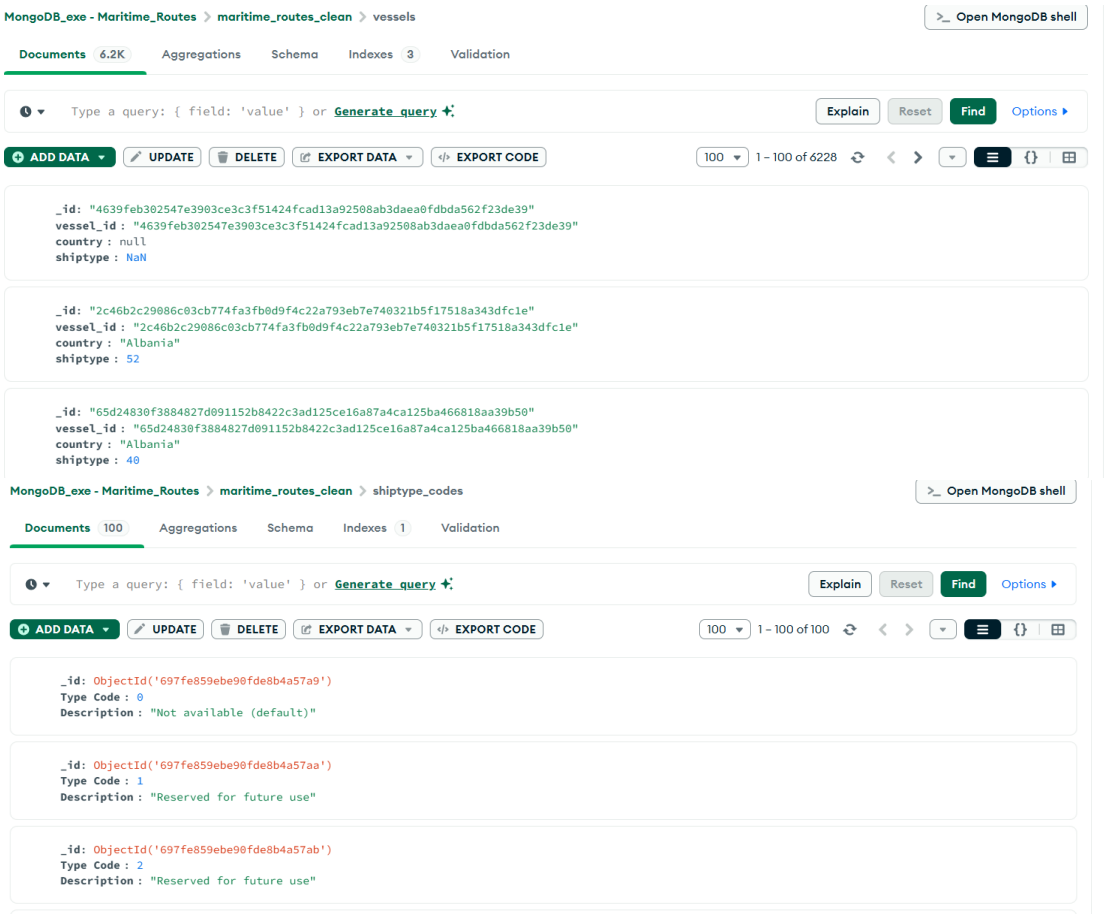
Κάθε έγγραφο στη συλλογή vessels χρησιμοποιεί το vessel_id ως _id, εξασφαλίζοντας μοναδικότητα και πολύ γρήγορη ανάκτηση στατικών χαρακτηριστικών με βάση το πλοίο. Η επιλογή αυτή είναι ιδιαίτερα χρήσιμη κατά την κατασκευή των τροχιών, όπου απαιτείται συχνό lookup της στατικής πληροφορίας για κάθε νέο vessel_id.

```
for d in docs:
    d["_id"] = d["vessel_id"]
vessels.insert_many(docs, ordered=False)
```

Για την υποστήριξη βασικών φίλτρων και ομαδοποιήσεων, δημιουργούνται επιπλέον ευρετήρια στα πεδία country και shiptype, εφόσον αυτά υπάρχουν στο dataset.

```
vessels.create_index([("country", ASCENDING)])
vessels.create_index([("shiptype", ASCENDING)])
```

Η παραπάνω σχεδίαση επιτρέπει τον καθαρό διαχωρισμό μεταξύ στατικής και δυναμικής πληροφορίας, μειώνει τον πλεονασμό δεδομένων και διευκολύνει τον εμπλουτισμό των τροχιών σε επίπεδο πλοίου χωρίς επαναλαμβανόμενα lookups κατά την εκτέλεση των ερωτημάτων.



4.3 Συλλογή weather_points (μετεωρολογικά δεδομένα)

Η συλλογή **weather_points** περιέχει μετεωρολογικές παρατηρήσεις με χωρική και χρονική αναφορά. Κάθε έγγραφο περιλαμβάνει GeoJSON Point για τη θέση, χρονική σήμανση και επιλεγμένα μετεωρολογικά μεγέθη (π.χ. θερμοκρασία, άνεμος). Τα δεδομένα αυτά δεν συνδέονται άμεσα με τα AIS σημεία μέσω κοινού κλειδιού, αλλά χρησιμοποιούνται για χωροχρονικό εμπλουτισμό κατά τη φάση κατασκευής των τροχιών.

Η επιλογή ξεχωριστής συλλογής για τα μετεωρολογικά δεδομένα επιτρέπει:

- ανεξάρτητη ευρετηρίαση (2dsphere + χρόνος),
- ευέλικτη αντιστοίχιση μέσω χωροχρονικών παραθύρων,
- και περιορισμό του payload που ενσωματώνεται στα τελικά trajectory documents.

4.3.1 Μεταφόρτωση και κανονικοποίηση μετεωρολογικών δεδομένων (weather_points)

Η συλλογή **weather_points** περιέχει μετεωρολογικές παρατηρήσεις με χωρική και χρονική αναφορά και χρησιμοποιείται για τον εμπλουτισμό των AIS σημείων κατά την κατασκευή τροχιών. Η μεταφόρτωση υλοποιείται με Python, αξιοποιώντας **GeoPandas** για ανάγνωση shapefile και **PyMongo** για εισαγωγή στη MongoDB. Στόχος του pipeline είναι: (i) η μετατροπή της γεωμετρίας σε GeoJSON Point κατάλληλο για 2dsphere index, (ii) η κανονικοποίηση του χρόνου σε UTC datetime, (iii) η διατήρηση των πρωτογενών πεδίων (raw) και (iv) η δημιουργία νέων, "human-friendly" πεδίων (derived) με μετατροπές μονάδων.

(α) Ανάγνωση shapefile και χειρισμός γεωμετρίας

Τα μετεωρολογικά δεδομένα παρέχονται ως shapefile. Κάθε εγγραφή διαθέτει γεωμετρία (geometry), η οποία μπορεί να είναι Point ή άλλη μορφή (π.χ. polygon/line). Για να υποστηριχθούν χωρικές αναζητήσεις στη MongoDB, όλες οι γεωμετρίες κανονικοποιούνται σε ένα αντιπροσωπευτικό **Point**, χρησιμοποιώντας το ίδιο το Point όταν υπάρχει ή το **centroid** όταν η γεωμετρία δεν είναι Point [5].

```
gdf = gpd.read_file(WEATHER_JAN2019_SHP)

geom = row.geometry
if geom.geom_type != "Point":
    geom = geom.centroid

doc["loc"] = {"type": "Point", "coordinates": [float(geom.x),
float(geom.y)]}
doc["lon"] = float(geom.x)
doc["lat"] = float(geom.y)
```

Με αυτόν τον τρόπο, εξασφαλίζεται κοινή αναπαράσταση για όλα τα records και δυνατότητα χρήσης 2dsphere index.

(β) Κανονικοποίηση χρόνου (epoch seconds → UTC datetime)

Στο shapefile ο χρόνος δίνεται ως epoch seconds στο πεδίο timestamp_. Για να μπορούν να γίνονται χρονικά range queries και συνδυαστικά φίλτρα με AIS σημεία, το πεδίο μετατρέπεται σε canonical timestamp τύπου datetime με timezone UTC.

```
def epoch_sec_to_dt_utc(sec: int) -> datetime:
    return datetime.fromtimestamp(sec, tz=timezone.utc)

ts_epoch = int(row.get("timestamp_"))
doc["timestamp_"] = ts_epoch
doc["timestamp"] = epoch_sec_to_dt_utc(ts_epoch)
```

Προαιρετικά διατηρείται και οποιοδήποτε string timestamp (αν υπάρχει) σε ξεχωριστό πεδίο timestamp_str, ώστε να μη χαθεί πληροφορία μορφοποίησης του αρχικού αρχείου.

(γ) Διατήρηση πρωτογενών πεδίων (raw) και παραγωγή νέων πεδίων (derived)

Το pipeline κρατά **όλα τα αρχικά πεδία** που παρέχονται από το shapefile (π.χ. TMP, PRMSL, WSPD, GUST, κ.λπ.) και επιπλέον δημιουργεί **νέα πεδία** με μετατροπές μονάδων, ώστε να διευκολυνθούν οι αναλύσεις και τα φίλτρα χωρίς να "αλλοιωθεί" η πρωτογενής πληροφορία.

Ενδεικτικά:

- Θερμοκρασίες: Kelvin → Celsius (tmp_c, tmin_c, tmax_c, dpt_c)
- Πίεση: Pascal → hPa (prmsl_hpa)
- Άνεμος: m/s → km/h (wspd_kmh, gust_kmh)
- Ορατότητα: meters → km (vis_km)
- Συνιστώσες ανέμου: m/s → km/h (ugrd_kmh, vgrd_kmh)

```
doc["tmp_c"]   = k_to_c(doc.get("TMP"))
doc["tmin_c"]  = k_to_c(doc.get("TMIN"))
doc["tmax_c"]  = k_to_c(doc.get("TMAX"))
doc["dpt_c"]   = k_to_c(doc.get("DPT"))

doc["prmsl_hpa"] = pa_to_hpa(doc.get("PRMSL"))

doc["wspd_kmh"] = ms_to_kmh(doc.get("WSPD"))
doc["gust_kmh"] = ms_to_kmh(doc.get("GUST"))

doc["vis_km"]  = m_to_km(doc.get("VIS"))
doc["ugrd_kmh"] = ms_to_kmh(doc.get("UGRD"))
doc["vgrd_kmh"] = ms_to_kmh(doc.get("VGRD"))
```

Για robustness, εφαρμόζονται "safe" helpers (_to_float, _round) ώστε αν κάποια τιμή λείπει ή δεν μετατρέπεται σε αριθμό, να αποθηκεύεται None χωρίς να αποτυγχάνει το import.

(δ) Rounding κανόνες και σταθεροποίηση τιμών

Κατά τη μεταφόρτωση εφαρμόζεται στρογγυλοποίηση τόσο στα raw όσο και στα derived πεδία, με στόχο να αποφεύγονται artefacts κινητής υποδιαστολής (π.χ. 1013.3652999999) και να μειώνεται το storage overhead. Οι κανόνες ορίζονται σε ενιαίο dictionary (ROUNDING_RULES) και εφαρμόζονται με τη συνάρτηση _apply_rounding(doc).

```
ROUNDING_RULES = {
    "TMP": 2, "TMIN": 2, "TMAX": 2, "DPT": 2,
    "PRMSL": 2, "WSPD": 2, "GUST": 2,
    "tmp_c": 2, "prmsl_hpa": 2, "wspd_kmh": 2, "gust_kmh": 2,
    "VIS": 0, "vis_km": 2
}
_apply_rounding(doc)
```

Σημειώνεται ότι **δεν εφαρμόζεται rounding** σε lon/lat/loc, ώστε να μην επηρεαστεί η χωρική ακρίβεια.

(ε) Batch εισαγωγή και δημιουργία ευρετηρίων

Η εισαγωγή γίνεται σε batches (π.χ. ανά 5.000 έγγραφα) ώστε να μειωθούν τα round-trips προς τη MongoDB και να βελτιωθεί η απόδοση της μεταφόρτωσης.

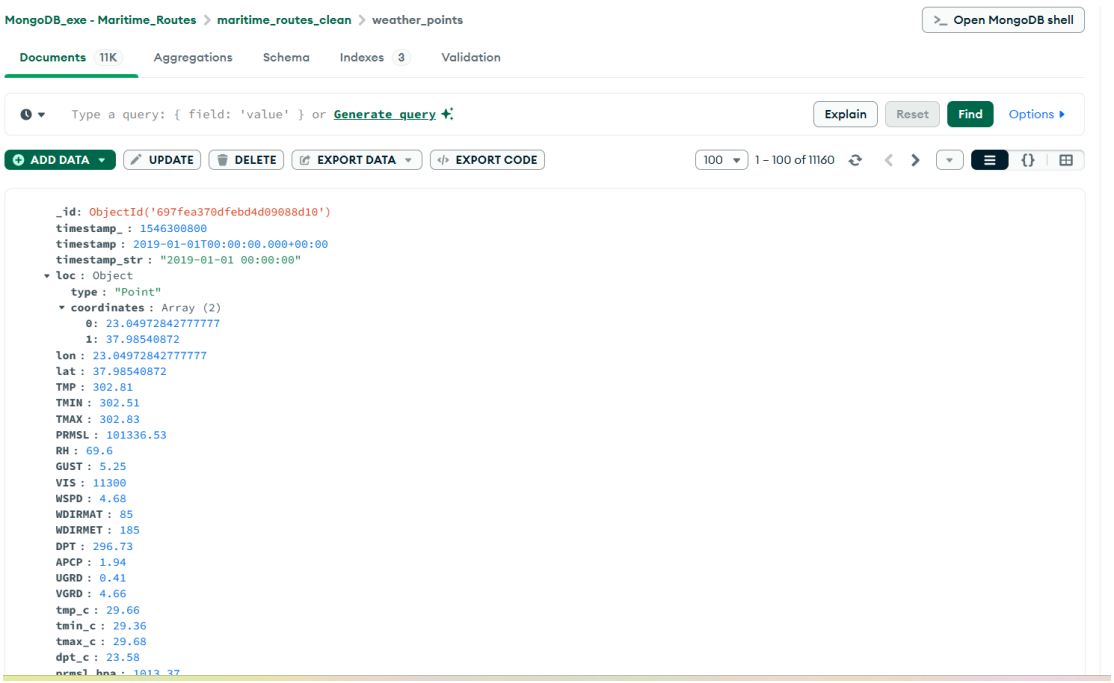
```
batch.append(doc)
if len(batch) >= 5000:
    coll.insert_many(batch, ordered=False)
    batch = []
```

Μετά την εισαγωγή δημιουργούνται τα βασικά ευρετήρια:

- **2dsphere** στο loc για χωρικά lookups,
- **index στο timestamp** για χρονικά φίλτρα και χωροχρονική αντιστοίχιση με AIS.

```
coll.create_index([("loc", "2dsphere")])
coll.create_index([("timestamp", 1)])
```

Η σχεδίαση αυτή επιτρέπει την αποδοτική εύρεση της πλησιέστερης μετεωρολογικής παρατήρησης στο build step (π.χ. με $near + χρονικό παράθυρο) και ταυτόχρονα διευκολύνει αναλύσεις και queries σε πιο εύχρηστες μονάδες.



4.4 Συλλογή trajectories (εμπλουτισμένες τροχιές)

Η συλλογή **trajectories** αποτελεί τον πυρήνα της βάσης δεδομένων. Κάθε έγγραφο αντιστοιχεί σε μία τροχιά πλοίου και περιλαμβάνει:

- το vessel_id,
- χρονικά όρια (start/end time),
- συνοπτικά χωρικά χαρακτηριστικά (centroid, bounding box),
- στατική πληροφορία πλοίου,
- και έναν πίνακα points, ο οποίος περιέχει τα επιμέρους AIS σημεία της τροχιάς.

Κάθε στοιχείο του πίνακα points περιλαμβάνει τη χωρική θέση, τη χρονική στιγμή, επιλεγμένα χαρακτηριστικά κίνησης, μετεωρολογική πληροφορία και ένα σύνολο annotations (π.χ. STOP, TURN, MISSING_COURSE). Η αποθήκευση των σημείων ως embedded array επιτρέπει την αποδοτική ανάκτηση πλήρους τροχιάς με ένα μόνο query και ευθυγραμμίζεται με τα ζητούμενα ερωτήματα της εργασίας, τα οποία αφορούν ανάλυση σε επίπεδο τροχιάς. Παράλληλα, για την υποστήριξη χωρικών φίλτρων σε επίπεδο τροχιάς, αποθηκεύεται το κεντροειδές της τροχιάς σε μορφή GeoJSON Point, το οποίο ευρετηριάζεται με 2dsphere index και χρησιμοποιείται για γρήγορο προ-φιλτράρισμα[4].

4.4.1 Κατασκευή εμπλουτισμένων τροχιών (trajectories) με segmentation, enrichment, annotations και χειρισμό ορίου 16MB

Η συλλογή **trajectories** δημιουργείται από τα πρωτογενή AIS σημεία (ais_points) μέσω ενός streaming pipeline που εκτελεί: (i) κατάτμηση (segmentation) των σημείων σε τροχιές με βάση χρονικά κενά, (ii) εμπλουτισμό κάθε τροχιάς με στατική πληροφορία πλοίου, (iii) εμπλουτισμό κάθε σημείου με μετεωρολογική πληροφορία μέσω χωροχρονικής αντιστοίχισης, (iv) παραγωγή annotations κίνησης ανά σημείο και (v) ασφαλή αποθήκευση με χειρισμό περιπτώσεων υπέρβασης του ορίου μεγέθους εγγράφου της MongoDB (16MB), με καταγραφή στη συλλογή failed_trajectories.

(α) Streaming επεξεργασία σημείων σε σειρά (αξιοποίηση index)

Η κατασκευή των τροχιών γίνεται με ανάγνωση όλων των AIS σημείων ταξινομημένων ανά vessel_id και timestamp. Η ταξινόμηση αυτή είναι κρίσιμη, καθώς επιτρέπει να επεξεργαστούμε τα δεδομένα ως ροή (stream) και να αποφύγουμε φόρτωση μεγάλων ομάδων στη μνήμη.

```
cursor = ais.find({}, {"_id": 0}).sort([("vessel_id", 1),
("timestamp", 1)])
for doc in cursor:
    vessel = doc["vessel_id"]
    ts = doc.get("timestamp")
    loc = doc.get("loc")
```

Η απόδοση του παραπάνω βήματος βασίζεται στο σύνθετο ευρετήριο (vessel_id, timestamp) που έχει δημιουργηθεί στη συλλογή ais_points.

(β) Segmentation τροχιών με κανόνα χρονικού κενού (GAP_HOURS)

Για να οριστεί πότε ξεκινά νέα τροχιά, εφαρμόζεται κανόνας κατάτμησης: νέα τροχιά δημιουργείται όταν αλλάζει το πλοίο (vessel_id) ή όταν το χρονικό κενό από το προηγούμενο σημείο υπερβαίνει το όριο GAP_HOURS (π.χ. 6 ώρες). Ο κανόνας αυτός αποτρέπει τη δημιουργία ψευδών τροχιών που θα "γεφύρωναν" μεγάλα χρονικά κενά.

```
new_traj = False
if prev_vessel is None:
    new_traj = True
elif vessel != prev_vessel:
    new_traj = True
```

```
elif prev_time is not None and ts is not None:
    if ts - prev_time > timedelta(hours=GAP_HOURS):
        new_traj = True
```

Όταν ξεκινά νέα τροχιά, η προηγούμενη «κλείνει» (υπολογίζονται spatial summaries και αποθηκεύεται στη MongoDB) και ανοίγει νέο trajectory document.

(γ) Enrichment σε επίπεδο τροχιάς με στατικά χαρακτηριστικά πλοίου (join μία φορά)

Για να αποφευχθεί επαναλαμβανόμενο "join" σε εκατομμύρια σημεία, η στατική πληροφορία πλοίου ανακτάται **μία φορά ανά τροχιά** από τη συλλογή vessels. Έτσι, στοιχεία όπως country και shiptype αποθηκεύονται στο trajectory document και είναι άμεσα διαθέσιμα στα ερωτήματα.

```
vdoc = db[COLL_VESSELS].find_one(
    {"vessel_id": vessel},
    {"_id": 0, "country": 1, "shiptype": 1}
)

current = {
    "vessel_id": vessel,
    "start_time": ts,
    "end_time": ts,
    "points": [],
    "vessel": {
        "country": vdoc.get("country") if vdoc else None,
        "shiptype": vdoc.get("shiptype") if vdoc else None,
    }
}
```

Η επιλογή αυτή μειώνει σημαντικά τα lookups κατά τη διάρκεια του build και περιορίζει τον πλεονασμό δεδομένων (δεν επαναλαμβάνεται η ίδια static πληροφορία σε κάθε point).

(δ) Εμπλουτισμός με καιρό ανά σημείο με χωροχρονικό παράθυρο + caching

Για κάθε AIS σημείο γίνεται αντιστοίχιση με την πλησιέστερη μετεωρολογική παρατήρηση εντός:

- χρονικού παραθύρου ± WEATHER_WINDOW_HOURS (π.χ. ±3 ώρες)
- χωρικής ακτίνας WEATHER_MAX_DISTANCE_METERS (π.χ. 50km)

Η αναζήτηση πραγματοποιείται με $near πάνω σε 2dsphere index και ταυτόχρονο φίλτρο στο timestamp.

```
q = {
  "loc": {"$near": {"$geometry": point_loc, "$maxDistance":
WEATHER_MAX_DISTANCE_METERS}},
  "timestamp": {"$gte": tmin, "$lte": tmax}
}
doc = wcoll.find_one(q, WEATHER_PROJECTION)
```

Επειδή ένα τέτοιο query ανά AIS σημείο είναι πολύ ακριβό (εκατομμύρια points), υλοποιείται **caching με χωροχρονικό bucketing**. Πολλά κοντινά σημεία μοιράζονται το ίδιο αποτέλεσμα weather lookup. Το cache key βασίζεται σε:

- bucket χρόνου (ανά 3 ώρες)
- bucket χώρου (lon/lat rounded σε 2 δεκαδικά ≈ ~1km)

```
time_bucket = floor_to_bucket_hours(ts, WEATHER_WINDOW_HOURS)
lon_bucket = round(lon, 2)
lat_bucket = round(lat, 2)
key = (time_bucket, lon_bucket, lat_bucket)
```

Αν το key υπάρχει στο cache, δεν εκτελείται νέο query. Επιπλέον, αποθηκεύεται και None όταν δεν υπάρχει αποτέλεσμα, ώστε να αποφεύγονται επαναλαμβανόμενα αποτυχημένα lookups.

(ε) Annotations ανά σημείο (STOP/SLOW/NORMAL/FAST, TURN, MISSING_*)

Για την υποστήριξη ερωτημάτων που βασίζονται σε "συμπεριφορικά" χαρακτηριστικά, παράγονται annotations ανά σημείο. Η παραγωγή γίνεται με κανόνες:

- STOP/SLOW/NORMAL/FAST από thresholds στην ταχύτητα
- MISSING_HEADING όταν λείπει heading
- MISSING_COURSE όταν λείπει course
- TURN όταν η γωνιακή διαφορά διαδοχικών course υπερβαίνει TURN_THRESHOLD_DEG

Η γωνιακή διαφορά υπολογίζεται με τρόπο που λαμβάνει υπόψη το κυκλικό πεδίο 0–360°:

```
def angular_diff_deg(a, b):
    diff = abs(a - b)
    return min(diff, 360 - diff)
Και τα tags παράγονται ως λίστα:
annotations = build_annotations(speed, heading, course,
prev_course_in_traj)

point = {
    "timestamp": ts,
    "loc": loc,
    "speed": speed,
    "heading": heading,
    "course": course,
    "annotations": annotations,
    "weather": weather
}
current["points"].append(point)
```

Σημαντική λεπτομέρεια είναι ότι το prev_course_in_traj μηδενίζεται στην αρχή κάθε τροχιάς, ώστε οι στροφές να υπολογίζονται μόνο μέσα στην ίδια τροχιά.

(στ) Συνοπτικά χωρικά χαρακτηριστικά τροχιάς (bbox, centroid)

Για κάθε τροχιά υπολογίζεται bounding box από τα coordinates των σημείων και παράγεται centroid ως το κέντρο του bbox. Το centroid αποθηκεύεται ως GeoJSON Point και χρησιμοποιείται αργότερα για χωρικά φίλτρα σε επίπεδο τροχιάς.

```
b = bbox_from_coords(coords_for_bbox)
current["bbox"] = b
current["centroid"] = centroid_from_bbox(b)
current["num_points"] = len(current["points"])
```

(ζ) Χειρισμός ορίου 16MB και καταγραφή αποτυχιών στη failed_trajectories

Η MongoDB επιβάλλει μέγιστο μέγεθος εγγράφου 16MB. Σε σπάνιες περιπτώσεις, μια τροχιά με πολλά σημεία (και embedded weather info) μπορεί να υπερβεί αυτό το όριο. Για να μη χαθεί πληροφορία και να υπάρχει ιχνηλασιμότητα, η εισαγωγή γίνεται μέσω helper flush_trajectory, η οποία σε περίπτωση DocumentTooLarge καταγράφει τη τροχιά στη συλλογή failed_trajectories με βασικά μεταδεδομένα.

```
try:
    tcoll.insert_one(traj_doc)
except DocumentTooLarge:
    failed_coll.insert_one({
        "reason": "DocumentTooLarge",
        "vessel_id": traj_doc.get("vessel_id"),
        "start_time": traj_doc.get("start_time"),
        "end_time": traj_doc.get("end_time"),
        "num_points": traj_doc.get("num_points"),
    })
```

(η) Ευρετηρίαση στη συλλογή trajectories

Μετά την κατασκευή των τροχιών, δημιουργούνται βασικά ευρετήρια που υποστηρίζουν τα ζητούμενα queries:

- index σε vessel_id για ανάκτηση τροχιών ενός πλοίου
- index σε start_time για χρονικά φίλτρα
- 2dsphere index σε centroid για χωρικά φίλτρα σε επίπεδο τροχιάς

```
traj_coll.create_index([("vessel_id", 1)])
traj_coll.create_index([("start_time", 1)])
traj_coll.create_index([("centroid", "2dsphere")])
```

Με τη συγκεκριμένη διαδικασία, η βάση αποκτά τελικά documents τροχιών που είναι **πλούσια σε πληροφορία (enriched)**, αλλά παραμένουν **query-friendly**, καθώς συνδυάζουν χωρική/χρονική πληροφορία, συμπεριφορικά annotations και επιλεγμένα μετεωρολογικά πεδία με τρόπο που υποστηρίζει αποδοτικά τα ερωτήματα της εργασίας.

4.4.2 Bucketing/catching

Η αντιστοίχιση καιρού σε κάθε AIS σημείο απαιτεί χωροχρονικό query (γεωχωρικό $near + χρονικό φίλτρο). Αν αυτό εκτελούνταν για **κάθε** σημείο, το κόστος θα ήταν δυσανάλογο, καθώς τα AIS σημεία είναι εκατομμύρια και θα οδηγούσαν σε εκατομμύρια κλήσεις προς τη συλλογή weather_points. Για να αντιμετωπιστεί αυτό, στο στάδιο κατασκευής των τροχιών υλοποιήθηκε **caching με χωροχρονικό bucketing**. Η βασική ιδέα είναι ότι πολλά AIS σημεία που είναι **κοντά στον χώρο και στον χρόνο** αναμένεται να έχουν την ίδια (ή σχεδόν ίδια) «καλύτερη» μετεωρολογική αντιστοίχιση. Έτσι, αντί να κάνουμε lookup για κάθε σημείο, ορίζουμε χωροχρονικά "κελιά" (buckets) και αποθηκεύουμε στο cache το αποτέλεσμα lookup για κάθε κελί. Στον κώδικα, ο χρόνος "κόβεται" σε blocks των 3 ωρών (WEATHER_WINDOW_HOURS = 3) μέσω floor_to_bucket_hours, ενώ ο χώρος κβαντίζεται με στρογγυλοποίηση του lon/lat σε 2 δεκαδικά, που αντιστοιχεί χονδρικά σε χωρική ακρίβεια της τάξης του ~1km. Το cache key είναι το τρίπτυχο:

- time_bucket (π.χ. 2019-01-01 12:00:00 UTC)
- lon_bucket (round(lon, 2))
- lat_bucket (round(lat, 2))

Αν ένα AIS σημείο πέσει στο ίδιο κελί με κάποιο προηγούμενο, επιστρέφεται άμεσα το cached αποτέλεσμα (ακόμη και αν είναι None), χωρίς νέο query προς τη MongoDB. Με αυτόν τον μηχανισμό μειώνεται σημαντικά το πλήθος των γεωχωρικών αναζητήσεων και βελτιώνεται η συνολική κλιμακωσιμότητα του build step, ειδικά όταν το dataset μεγαλώνει.

Πιθανή κατάτμηση (sharding) για περαιτέρω κλιμάκωση

Για ακόμη μεγαλύτερη κλίμακωση, μια φυσική επέκταση της λύσης θα ήταν η χρήση **MongoDB sharding** σε cluster (πολλαπλά shards + config servers + mongos routers). Σε ένα τέτοιο σενάριο, οι συλλογές μεγάλου όγκου θα μπορούσαν να κατατμηθούν ώστε η αποθήκευση και τα queries να διαμοιράζονται σε πολλούς κόμβους.

Μια ρεαλιστική επιλογή shard key για τα δεδομένα της εργασίας θα ήταν:

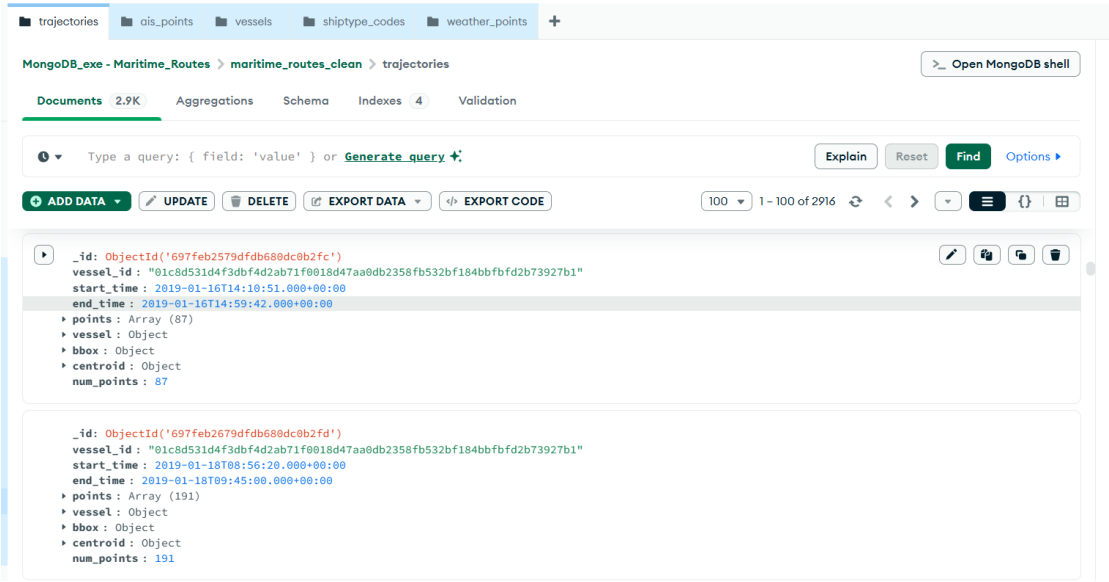
- για ais_points: σύνθετο shard key όπως **{ vessel_id: 1, timestamp: 1 }**, ώστε τα δεδομένα να κατανέμονται ανά πλοίο, ενώ ταυτόχρονα να υποστηρίζονται range queries στον χρόνο (με την προϋπόθεση ότι το key έχει επαρκή κατανομή).
- εναλλακτικά, shard key με **hashed vessel_id** (π.χ. { vessel_id: "hashed" }) για καλύτερη ισορροπία φορτίου (load balancing), με trade-off ότι τα καθαρά time-range queries δεν ευνοούνται όσο με range-based key.

Για τη συλλογή trajectories, αντίστοιχα, πιθανό shard key θα μπορούσε να είναι είτε:

- { vessel_id: 1, start_time: 1 } (ευνοεί queries ανά πλοίο/χρόνο), είτε { vessel_id: "hashed" } (ευνοεί ομοιόμορφη κατανομή τροχιών).

Στην παρούσα εργασία δεν υλοποιήθηκε sharded cluster, καθώς απαιτεί επιπλέον υποδομή (πολλαπλά containers/VMs, config servers, routers, tuning balancer) και εκτεταμένη πειραματική αξιολόγηση σε πολλαπλά σενάρια. Επειδή ο διαθέσιμος χρόνος επικεντρώθηκε στην τεκμηριωμένη μοντελοποίηση, στην αποδοτική ευρετηρίαση και στην υλοποίηση των ζητούμενων ερωτημάτων/μετρήσεων, η λύση αξιολογήθηκε σε single-node MongoDB setup.

Παρ’ όλα αυτά, ο σχεδιασμός (ιδίως τα indexes και ο διαχωρισμός συλλογών) είναι συμβατός με μελλοντική μετάβαση σε sharding, εφόσον απαιτηθεί μεγαλύτερη κλίμακωση.



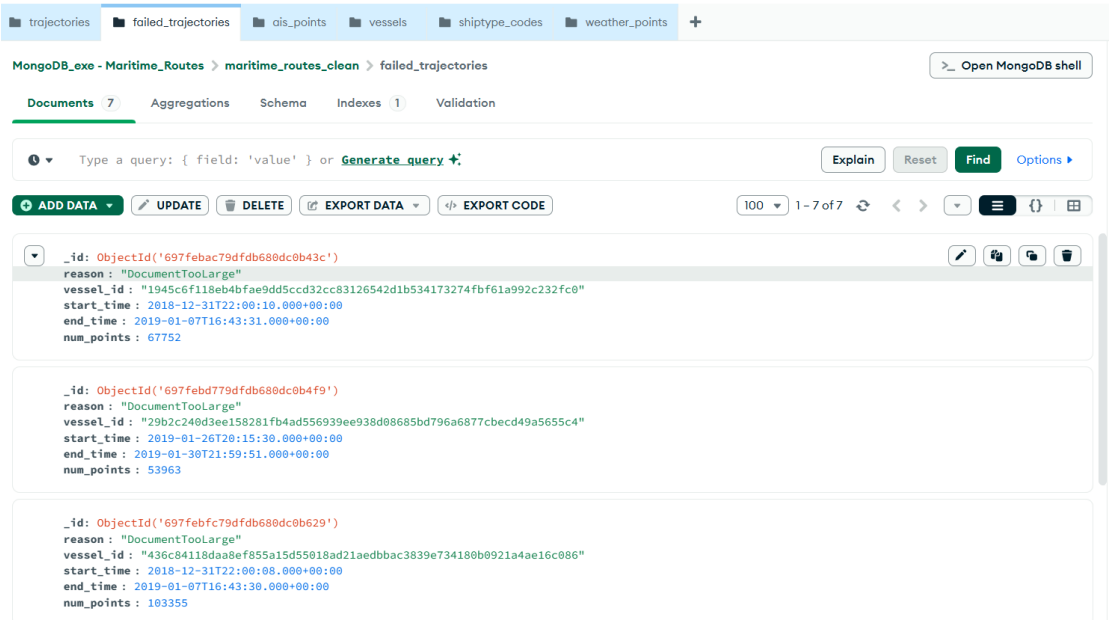

```
{
  "_id": "697feb2579dfdb680dc0b2fc",
  "vessel_id": "01c8d531d4f3dbf4d2ab71f0018d47aa0db2358fb532bf184bbfbfd2b73927b1",
  "start_time": "2019-01-16T14:10:51.000Z",
  "end_time": "2019-01-16T14:59:42.000Z",
  "points": [
    {
      "timestamp": "2019-01-16T14:10:51.000Z",
      "loc": {
        "type": "Point",
        "coordinates": [23.55045, 37.95785]
      },
      "speed": 9.5,
      "heading": 104,
      "course": 135.3,
      "annotations": ["NORMAL"],
      "weather": {
        "timestamp": "2019-01-16T15:00:00.000Z",
        "RH": 69.2,
        "WDIRMET": 290.6,
        "APCP": 0,
        "tmp_c": 30.02,
        "prmsl_hpa": 1009.47,
        "wspd_kmh": 12.78,
        "gust_kmh": 12.41
      }
    },
    {
      "timestamp": "2019-01-16T14:12:00.000Z",
      "loc": {
        "type": "Point",
        "coordinates": [23.55355, 37.95605]
      },
      "speed": 9.5,
      "heading": 98,
      "course": 121.3,
      "annotations": ["NORMAL"],
      "weather": {
```

```
        "timestamp": "2019-01-16T15:00:00.000Z",
        "RH": 69.2,
        "WDIRMET": 290.6,
        "APCP": 0,
        "tmp_c": 30.02,
        "prmsl_hpa": 1009.47,
        "wspd_kmh": 12.78,
        "gust_kmh": 12.41
      }
    },
    {
      "timestamp": "2019-01-16T14:13:00.000Z",
      "loc": {
        "type": "Point",
        "coordinates": [23.55645, 37.954733333333294]
      },
      "speed": 9.4,
      "heading": 96,
      "course": 118.8,
      "annotations": ["NORMAL"],
      "weather": {
        "timestamp": "2019-01-16T15:00:00.000Z",
        "RH": 69.2,
        "WDIRMET": 290.6,
        "APCP": 0,
        "tmp_c": 30.02,
        "prmsl_hpa": 1009.47,
        "wspd_kmh": 12.78,
        "gust_kmh": 12.41
      }
    }
  ]
}
```


4.5 Συλλογή failed_trajectories και περιορισμοί πλατφόρμας

Κατά την κατασκευή των τροχιών λαμβάνεται υπόψη ο περιορισμός μεγέθους εγγράφου της MongoDB (16MB). Σε περιπτώσεις όπου μια τροχιά υπερβαίνει το όριο αυτό, η εισαγωγή της αποτυγχάνει και τα σχετικά μεταδεδομένα καταγράφονται στη συλλογή **failed_trajectories**. Η συλλογή αυτή περιλαμβάνει πληροφορίες όπως vessel_id, χρονικά όρια και πλήθος σημείων, επιτρέποντας την ανάλυση και τεκμηρίωση ακραίων περιπτώσεων.

Η ύπαρξη της συλλογής failed_trajectories τεκμηριώνει ότι ο σχεδιασμός της βάσης λαμβάνει υπόψη πρακτικούς περιορισμούς της πλατφόρμας και παρέχει σαφή μηχανισμό ιχνηλασιμότητας, χωρίς απώλεια πληροφορίας από τη διαδικασία επεξεργασίας.



4.6 Σύνοψη σχεδιαστικών επιλογών

Συνοψίζοντας, η μοντελοποίηση της βάσης δεδομένων βασίζεται:

- σε **trajectory-centric σχεδίαση**,
- σε συνδυασμό **embedding** (points, annotations) και **referencing** (raw AIS, weather, static),
- και σε σαφή διαχωρισμό μεταξύ πρωτογενών δεδομένων και παραγόμενων αποτελεσμάτων.

Οι επιλογές αυτές έγιναν με γνώμονα την αποδοτική υποστήριξη χωροχρονικών και συναθροιστικών ερωτημάτων, τη μείωση της πολυπλοκότητας κατά την εκτέλεση queries και την κλιμακωσιμότητα της λύσης.

5.1 Σύνθετο ερώτημα αναζήτησης τροχιών (A2.1)

Για την κάλυψη της απαίτησης A2.1 υλοποιήθηκε ένα σύνθετο ερώτημα στη MongoDB που αναζητά τροχιές πλοίων με ταυτόχρονη χρήση **χωρικών**, **χρονικών**, **μετεωρολογικών** και **συμπεριφορικών (annotations)** φίλτρων. Το ερώτημα λειτουργεί σε επίπεδο εγγράφων της συλλογής trajectories και αξιοποιεί τόσο τα μεταδεδομένα τροχιάς (χρόνος έναρξης/λήξης, κεντροειδές) όσο και το embedded array points.

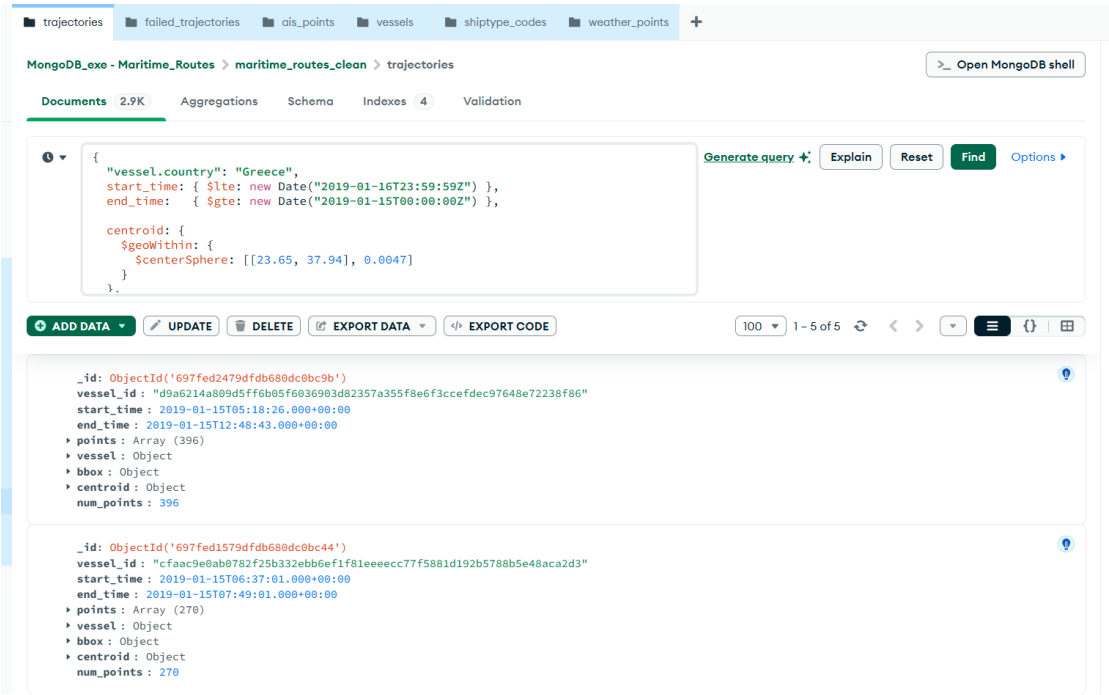
Συγκεκριμένα, το παρακάτω query επιστρέφει τροχιές πλοίων ελληνικής σημαίας (vessel.country = "Greece") που έχουν χρονική επικάλυψη με το διάστημα **[2019-01-15, 2019-01-16]**, των οποίων το **κεντροειδές** βρίσκεται εντός κύκλου ακτίνας ~30km γύρω από συντεταγμένες ενδιαφέροντος (περιοχή Πειραιά), και οι οποίες ικανοποιούν δύο επιπλέον κριτήρια στο επίπεδο των σημείων: (i) περιέχουν τουλάχιστον ένα σημείο με annotation **STOP**, και (ii) περιέχουν τουλάχιστον ένα σημείο με μετεωρολογική συνθήκη **weather.wspd_kmh ≥ 20**. Σημειώνεται ότι τα δύο τελευταία κριτήρια αφορούν την **ίδια**

τροχιά, χωρίς να απαιτείται να ισχύουν στο ίδιο ακριβώς σημείο, κάτι που είναι συμβατό με ερωτήματα τύπου “η τροχιά περιλαμβάνει στάση και εκτέθηκε σε ισχυρό άνεμο σε κάποιο τμήμα της”.

```
{
  "vessel.country": "Greece",
  start_time: { $lte: new Date("2019-01-16T23:59:59Z") },
  end_time:   { $gte: new Date("2019-01-15T00:00:00Z") },

  centroid: {
    $geoWithin: {
      $centerSphere: [[23.65, 37.94], 0.0047]
    }
  },

  $and: [
    { points: { $elemMatch: { annotations: "NORMAL" } } },
    { points: { $elemMatch: { "weather.wspd_kmh": { $gte: 20 } } } }
  ]
}
```

Τεκμηρίωση αποδοτικότητας.

Αρχικά, εφαρμόζονται φίλτρα σε επίπεδο τροχιάς (χρονική επικάλυψη με start_time/end_time και χωρικό φίλτρο στο centroid), τα οποία μειώνουν δραστικά τον αριθμό υποψήφιων εγγράφων που θα εξεταστούν. Στη συνέχεια, εφαρμόζονται οι συνθήκες πάνω στο array points μέσω $elemMatch, ώστε να ελεγχθεί αν η τροχιά περιέχει σημεία με συγκεκριμένα annotations και μετεωρολογικά χαρακτηριστικά. Η συλλογή trajectories διαθέτει 2dsphere ευρετήριο στο centroid, που επιταχύνει τα χωρικά φίλτρα, καθώς και ευρετήριο στο start_time για χρονικά range queries. Με αυτόν τον τρόπο, το “βαρύ” κομμάτι του ελέγχου (πάνω στο embedded array) εκτελείται μόνο για περιορισμένο σύνολο εγγράφων, διατηρώντας το query αποδοτικό και κλιμακώσιμο.



5.2 Συναθροιστικό ερώτημα A2.2: Πλήθος τροχιών σε χωρο-χρονικό κουτί

Για την απαίτηση A2.2 υλοποιήθηκε aggregation pipeline που καταμετρά το πλήθος των τροχιών οι οποίες **εμφανίζονται εντός συγκεκριμένου χωρο-χρονικού κουτιού**. Ορίζουμε ότι μια τροχιά "βρέθηκε εντός" του κουτιού όταν περιέχει **τουλάχιστον ένα σημείο** του οποίου (i) η γεωγραφική θέση βρίσκεται εντός χωρικού bounding box και (ii) ο χρόνος βρίσκεται εντός του επιλεγμένου χρονικού παραθύρου. Στην εργασία χρησιμοποιήθηκε χωρικό κουτί με όρια lon ∈ [23.50, 23.75] και lat ∈ [37.85, 38.05] (ευρύτερη περιοχή Πειραιά) και χρονικό παράθυρο μίας ημέρας (2019-01-15).

Η υλοποίηση γίνεται με $unwind στο embedded array points ώστε κάθε σημείο να εξεταστεί ανεξάρτητα, ακολουθεί χωρικό φιλτράρισμα σε επίπεδο σημείου, και στη συνέχεια μετατροπή του timestamp σε ημερομηνία ($dateToString) για να εφαρμοστεί φίλτρο ημέρας. Για να μετρηθεί κάθε τροχιά μία φορά (και όχι κάθε σημείο), γίνεται $group με κλειδί το _id της τροχιάς και τέλος $count. Με αυτό τον τρόπο, το τελικό αποτέλεσμα αντιστοιχεί σε **αριθμό τροχιών** και όχι αριθμό σημείων.

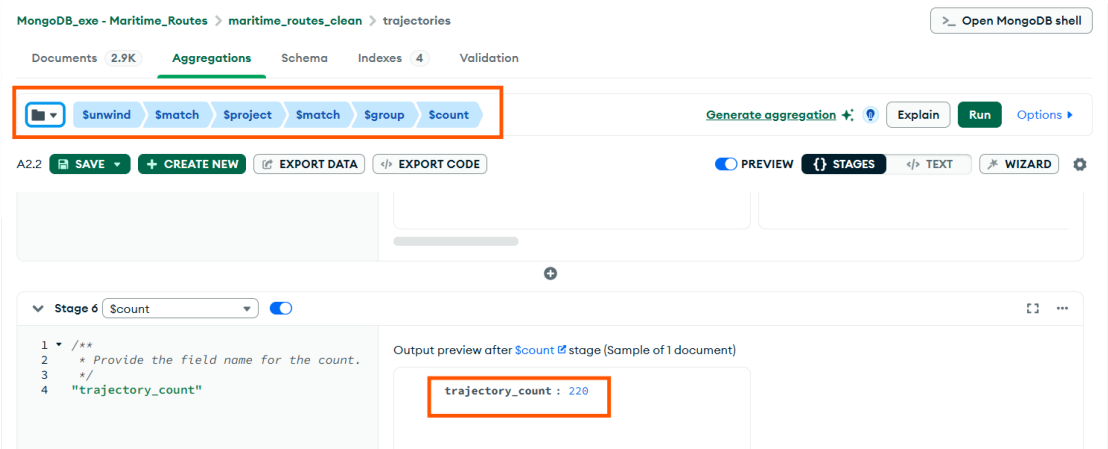
Αποτέλεσμα: Για τα δεδομένα του συστήματος, στο παραπάνω χωρο-χρονικό κουτί βρέθηκαν **220 τροχιές**.

Aggregation pipeline

```
[
  { "$unwind": "$points" },

  { "$match": {
      "points.loc.coordinates.0": { "$gte": 23.50, "$lte": 23.75 },
      "points.loc.coordinates.1": { "$gte": 37.85, "$lte": 38.05 }
  }},

  { "$project": {
```

```
      "date": {
        "$dateToString": {
          "format": "%Y-%m-%d",
          "date": "$points.timestamp"
        }
      }
  }},

  { "$match": { "date": "2019-01-15" } },

  { "$group": { "_id": "$_id" } },

  { "$count": "trajectory_count" }
]
```

- Το $unwind επιτρέπει ακριβή έλεγχο χωρο-χρονικών κριτηρίων σε επίπεδο σημείου, κάτι που είναι απαραίτητο όταν το “εντός κουτιού” ορίζεται ως ύπαρξη έστω ενός point.
- Η ομαδοποίηση ($group) με κλειδί το _id εξασφαλίζει ότι κάθε τροχιά μετράται μία φορά, ανεξάρτητα από το πόσα points της βρίσκονται εντός της περιοχής.
- Το pipeline είναι επεκτάσιμο: μπορούν να προστεθούν επιπλέον φίλτρα (π.χ. annotations ή weather) πριν το $group χωρίς να αλλάζει ο τρόπος καταμέτρησης.



6. Πειραματική αξιολόγηση

6.1 Πειραματικό περιβάλλον και μεθοδολογία

Η πειραματική αξιολόγηση πραγματοποιήθηκε πάνω στη συλλογή trajectories. Οι δοκιμές εκτελέστηκαν μέσω του MongoDB Compass, χρησιμοποιώντας aggregation pipelines. Για τη μέτρηση της απόδοσης καταγράφηκαν οι μετρικές που παρέχει το **Explain Plan** (Query Performance Summary), και συγκεκριμένα: (i) ο συνολικός χρόνος εκτέλεσης (*execution time* σε ms), (ii) το πλήθος εγγράφων που εξετάστηκαν (*documents examined*), και (iii) αν χρησιμοποιήθηκε ευρετήριο ή αν έγινε σάρωση συλλογής (COLLSCAN). Στόχος είναι να αξιολογηθεί η συμπεριφορά των ερωτημάτων υπό διαφορετικά επίπεδα πολυπλοκότητας και να αναδειχθούν τα trade-offs της embedded μοντελοποίησης (απλούστερα queries σε επίπεδο τροχιάς, με κόστος αυξημένου processing όταν απαιτείται $unwind) [4].

6.2 Ερώτημα αναφοράς (baseline) και ορισμός χωρο-χρονικού κουτιού

Ως ερώτημα αναφοράς (baseline) επιλέχθηκε το aggregation που αντιστοιχεί στην απαίτηση A2.2: καταμέτρηση τροχιών που “εμφανίζονται” εντός χωρο-χρονικού κουτιού. Το χωρικό φίλτρο ορίζεται ως bounding box με lon ∈ [23.50, 23.75] και lat ∈ [37.85, 38.05], ενώ το χρονικό φίλτρο αντιστοιχεί στην ημέρα 2019-01-15. Μια τροχιά θεωρείται ότι ανήκει στο χωρο-χρονικό κουτί όταν περιέχει **τουλάχιστον ένα σημείο** (point) που ικανοποιεί τα χωρικά κριτήρια και έχει timestamp εντός της συγκεκριμένης ημερομηνίας. Για τον λόγο αυτό, το pipeline κάνει $unwind στο array points, εφαρμόζει το χωρικό φίλτρο σε επίπεδο σημείου, μετατρέπει τον χρόνο σε ημερομηνία ($dateToString), και στη συνέχεια με $group εξασφαλίζει ότι κάθε τροχιά μετράται μία φορά (unique _id) πριν το τελικό $count.

Baseline pipeline (Q1):

```
[
  { "$unwind": "$points" },

  { "$match": {
      "points.loc.coordinates.0": { "$gte": 23.50, "$lte": 23.75 },
      "points.loc.coordinates.1": { "$gte": 37.85, "$lte": 38.05 }
  }},

  { "$project": {
      "date": {
        "$dateToString": {
          "format": "%Y-%m-%d",
          "date": "$points.timestamp"
        }
      }
  }},

  { "$match": { "date": "2019-01-15" } },

  { "$group": { "_id": "$_id" } },

  { "$count": "trajectory_count" }
]
```

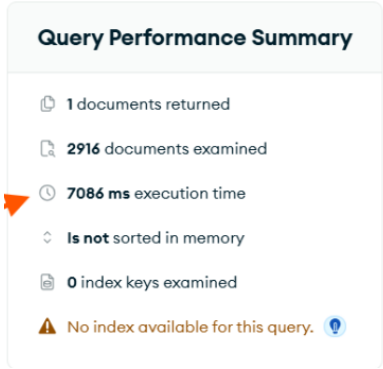
Το ερώτημα επέστρεψε μία εγγραφή με trajectory_count = 220, δηλαδή 220 διαφορετικές τροχιές που διέσχισαν το συγκεκριμένο χωρο-χρονικό κουτί.

6.3 Αποτελέσματα απόδοσης του baseline ερωτήματος

Για κάθε ερώτημα καταγράφονται: (α) *execution time* (ms), (β) *documents examined*, (γ) τύπος εκτέλεσης (COLLSCAN ή χρήση index).

Η ανάλυση μέσω του Explain Plan έδωσε τα ακόλουθα αποτελέσματα:

- **Execution time:** 7086 ms
- **Documents examined:** 2916
- **Documents returned:** 1
- **Index usage:** καμία (COLLSCAN)



Η εκτέλεση πραγματοποιήθηκε με πλήρη σάρωση της συλλογής (COLLSCAN), γεγονός αναμενόμενο δεδομένου ότι το $unwind σε embedded arrays ακυρώνει τη δυνατότητα άμεσης αξιοποίησης απλών ευρετηρίων σε επίπεδο τροχιάς (π.χ. centroid ή start_time). Παρ' όλα αυτά, το aggregation ολοκληρώνεται σε αποδεκτό χρόνο για αναλυτικού τύπου ερωτήματα, επιβεβαιώνοντας ότι η επιλεγμένη μοντελοποίηση είναι κατάλληλη για offline analytics.

6.4 Πείραμα Q2 – Pipeline (bbox + date + STOP)

Για να αξιολογηθεί η επίδραση της πολυπλοκότητας του ερωτήματος, συγκρίθηκαν δύο aggregation pipelines που έχουν **ίδιο χωρικό φίλτρο** (bbox γύρω από Πειραιά) και **ίδιο χρονικό φίλτρο** (ημέρα 2019-01-15), αλλά διαφέρουν ως προς το αν περιλαμβάνουν φίλτρο συμπεριφοράς (annotations).

- **Q1 (baseline):** bbox + ημερομηνία
- **Q2:** bbox + ημερομηνία + points.annotations = "STOP"

Και τα δύο queries υλοποιούνται με $unwind στο points, ώστε τα φίλτρα να εφαρμόζονται σε επίπεδο σημείου.

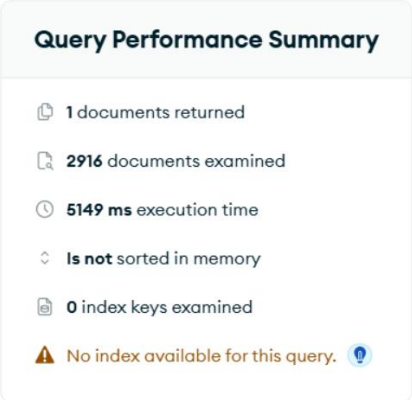
```
[
  { "$unwind": "$points" },
```

```
  { "$match": {
      "points.loc.coordinates.0": { "$gte": 23.50, "$lte": 23.75 },
      "points.loc.coordinates.1": { "$gte": 37.85, "$lte": 38.05 },
      "points.annotations": "STOP"
  }},

  { "$project": {
      "date": {
        "$dateToString": { "format": "%Y-%m-%d", "date":
"$points.timestamp" }
      }
  }},

  { "$match": { "date": "2019-01-15" } },

  { "$group": { "_id": "$_id" } },

  { "$count": "trajectory_count" }
]
```

Στο δεύτερο query προστίθεται φίλτρο πάνω στο πεδίο points.annotations, επιλέγοντας μόνο σημεία που φέρουν την ετικέτα **STOP**.

Αποτελέσματα Explain Plan:

- Documents examined: **2916**
- Index keys examined: **0**
- Execution time: **5149 ms**
- Execution strategy: **COLLSCAN**



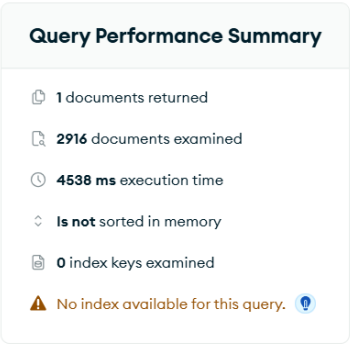
Παρατήρηση:
Παρότι το query είναι λογικά πιο σύνθετο, ο χρόνος εκτέλεσης μειώνεται σε σχέση με το Q1. Αυτό οφείλεται στο γεγονός ότι το φίλτρο STOP εφαρμόζεται νωρίς στο pipeline, μειώνοντας τον αριθμό των point-documents που προωθούνται στα επόμενα στάδια ($project, $group).

6.5 Πλήρες σύνθετο ερώτημα Q3 – Pipeline (bbox + date + STOP + wind)

Το επόμενο πείραμα εξετάζει την επιβάρυνση όταν, πέρα από χωρικό/χρονικό φίλτρο και annotation, εισάγεται και **μετεωρολογικός περιορισμός** σε επίπεδο σημείου. Αυτό προσεγγίζει άμεσα την απαίτηση A2.1, όπου ζητείται συνδυασμός χωρικών φίλτρων, φίλτρων σε annotations και φίλτρων σε weather.

Σημείωση: προτείνεται αρχικά κατώφλι ανέμου >= 15 km/h ώστε να υπάρχουν αποτελέσματα (αν βγάλει πολύ λίγα, μπορεί να πέσει σε 10· αν βγάλει πολλά, μπορεί να ανέβει σε 20).

```
[
  { "$unwind": "$points" },

  { "$match": {
      "points.loc.coordinates.0": { "$gte": 23.50, "$lte": 23.75 },
      "points.loc.coordinates.1": { "$gte": 37.85, "$lte": 38.05 },
      "points.annotations": "STOP",
      "points.weather.wspd_kmh": { "$gte": 15 }
  }},

  { "$project": {
      "date": {
        "$dateToString": { "format": "%Y-%m-%d", "date":
"$points.timestamp" }
      }
  }},

  { "$match": { "date": "2019-01-15" } },

  { "$group": { "_id": "$_id" } },

  { "$count": "trajectory_count" }
]
```



Αποτελέσματα Explain Plan (Q3):

- Documents examined: **2916**
- Index keys examined: **0**
- Execution time: **4538 ms**
- Execution strategy: **COLLSCAN**

6.6 Επιπλέον σύνθετο ερώτημα Q4 – Pipeline (bbox + date + STOP + TURN + wind)

Για να διερευνηθεί περαιτέρω η συμπεριφορά του συστήματος σε ακόμη πιο απαιτητικά αναλυτικά ερωτήματα, σχεδιάστηκε ένα επιπλέον query (Q4), το οποίο επεκτείνει το Q3 εισάγοντας και **συμπεριφορικό φίλτρο αλλαγής πορείας (TURN)**.

Το ερώτημα αυτό στοχεύει στον εντοπισμό τροχιών που:

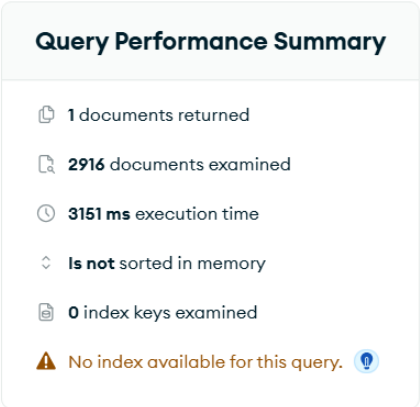
- περνούν από συγκεκριμένο χωρικό παράθυρο (bbox),
- περιέχουν σημεία συγκεκριμένης ημέρας,
- εμφανίζουν στάση (STOP),
- παρουσιάζουν έντονη αλλαγή πορείας (TURN),
- και ταυτόχρονα εκτελούνται υπό συνθήκες αυξημένου ανέμου.

Pipeline Q4

```
[
  { "$unwind": "$points" },

  { "$match": {
      "points.loc.coordinates.0": { "$gte": 23.50, "$lte": 23.75 },
      "points.loc.coordinates.1": { "$gte": 37.85, "$lte": 38.05 },
      "points.annotations": { "$all": ["STOP", "TURN"] },
      "points.weather.wspd_kmh": { "$gte": 15 }
  }},

  { "$project": {
      "date": {
        "$dateToString": {
          "format": "%Y-%m-%d",
          "date": "$points.timestamp"
        }
      }
  }},

  { "$match": { "date": "2019-01-15" } },

  { "$group": { "_id": "$_id" } },

  { "$count": "trajectory_count" }
]
```

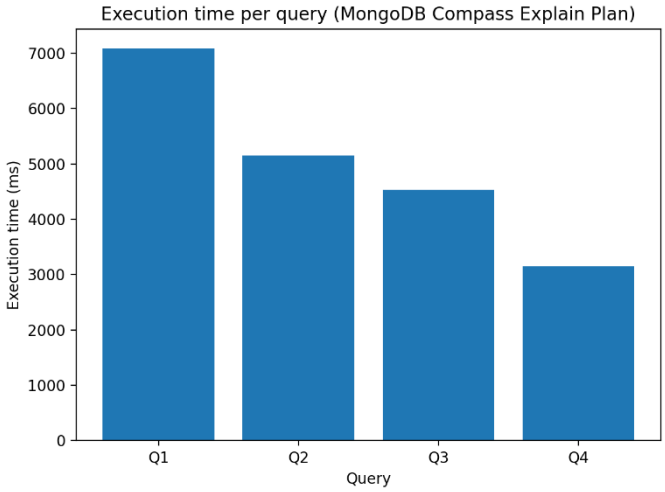
Αποτελέσματα Explain Plan (Q4):

- **Documents examined:** 2916
- **Index keys examined:** 0
- **Execution time:** 3151 ms
- **Execution strategy:** COLLSCAN



6.7 Συγκριτικός Πίνακας Αποτελεσμάτων

Query	Περιγραφή	Execution Time (ms)	Documents Examined	Index Usage
Q1	bbox + date	7086	2916	COLLSCAN
Q2	bbox + date + STOP	5149	2916	COLLSCAN
Q3	bbox + date + STOP + wind	4538	2916	COLLSCAN
Q4	bbox + date + STOP + TURN + wind	3151	2916	COLLSCAN



6.8 Συζήτηση Αποτελεσμάτων

Από τη συγκριτική αξιολόγηση των τεσσάρων queries προκύπτουν τα ακόλουθα βασικά συμπεράσματα:

1. **Όλα τα ερωτήματα εκτελούνται με COLLSCAN**, καθώς η χρήση $unwind σε embedded arrays καθιστά μη άμεσα αξιοποιήσιμα τα κλασικά MongoDB indexes. Το σύστημα απαιτεί πλήρη σάρωση της συλλογής τροχιών.

2. **Η αύξηση της λογικής πολυπλοκότητας δεν συνεπάγεται γραμμική αύξηση χρόνου εκτέλεσης.** Αντίθετα, παρατηρείται σταδιακή μείωση του execution time από το Q1 στο Q4.

3. Το φαινόμενο αυτό εξηγείται από το γεγονός ότι **τα επιπλέον φίλτρα (STOP, TURN, wind)** είναι ιδιαίτερα επιλεκτικά και μειώνουν σημαντικά τον αριθμό των points που περνούν στα επόμενα στάδια του pipeline ($project, $group).

4. Το Q4, παρότι είναι το πιο σύνθετο λογικά ερώτημα, παρουσιάζει τον μικρότερο χρόνο εκτέλεσης, επιβεβαιώνοντας ότι **η επιλεκτικότητα των φίλτρων υπερισχύει της συντακτικής πολυπλοκότητας**.

5. Η ενσωμάτωση μετεωρολογικών δεδομένων και annotations σε επίπεδο σημείου επιτρέπει την έκφραση σύνθετων spatio-temporal και behavioral queries **χωρίς την ανάγκη joins**, κάτι που απλοποιεί σημαντικά τη λογική των ερωτημάτων.

6. Το μοντέλο με embedded points ευνοεί κυρίως **αναλυτικά (analytic) workloads**, όπου προέχει η εκφραστικότητα και όχι η αυστηρά index-based εκτέλεση.

6.9 Συμπέρασμα

Η πειραματική αξιολόγηση καταδεικνύει ότι το προτεινόμενο μοντέλο δεδομένων υποστηρίζει αποτελεσματικά σύνθετα ερωτήματα ανάλυσης τροχιών, συνδυάζοντας χωρικά, χρονικά, συμπεριφορικά και περιβαλλοντικά φίλτρα σε ενιαία pipelines. Παρά την απουσία index-based εκτέλεσης στα εξεταζόμενα queries, οι χρόνοι απόκρισης παραμένουν αποδεκτοί για datasets της τάξης μερικών χιλιάδων τροχιών. Το αποτέλεσμα αυτό επιβεβαιώνει τη χρησιμότητα της προσέγγισης για ερευνητικούς και αναλυτικούς σκοπούς, καθώς και τη σημασία της σωστής μοντελοποίησης και της επιλεκτικότητας των φίλτρων σε σύνθετα spatio-temporal queries.

[1] H. D. Montgomery and A. P. Turner, “AIS data: A systematic review of applications and data challenges,” *The Journal of Navigation*, vol. 74, no. 6, pp. 1305–1326, 2021. doi: 10.1017/S037346332100046X

[2] International Association of Marine Aids to Navigation and Lighthouse Authorities (IALA), *Guideline on AIS Data and Use*, IALA Guideline G1082, 2021.

[3] MongoDB Inc., *MongoDB Manual: Indexes and Geospatial Indexes*, 2024. Available: https://www.mongodb.com/docs/manual/indexes/

[4] MongoDB – Aggregation Pipeline & explain MongoDB Inc., *MongoDB Manual: Aggregation Pipeline and explain()*, 2024. Available: https://www.mongodb.com/docs/manual/aggregation/

[5] NOAA – National Oceanic and Atmospheric Administration (NOAA), *NOAA NCEP Reanalysis Data Documentation*, NOAA, 2020. Available: https://psl.noaa.gov/data/gridded/data.ncep.reanalysis.html